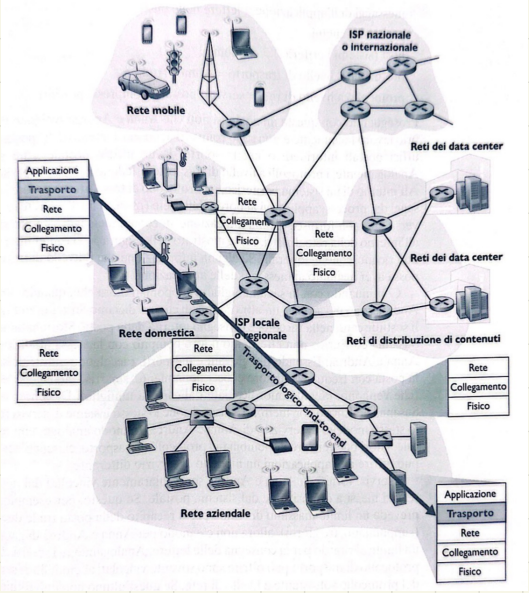
3.1 - Introduzione servizi a livello di trasporto



Un protocollo a livello di trasporto mette a disposizione una **comunicazione logica** (logical comunication) tra processi applicativi di host differenti, ovvero è come se, dal punto di vista dell'applicazione, gli host che eseguono i processi fossero direttamente connessi tra loro, quando in realtà tra di loro ci potrebbero essere svariati router e tipi di collegamenti, ma d'altronde i processi applicativi usano la comunicazione logica fornita dal livello di trasporto per scambiare i messaggi incuranti dell'enorme infrastruttura coinvolta per trasportarli

Come è possibile anche osservare dall'immagine, i protocolli a livello di trasporto sono implementati nei sistemi periferici, ma non nei router della rete

Adesso esaminiamo come avviene lo scambio di messaggi:

- dal lato mittente

  Il livello di trasporto converte i messaggi che riceve da un processo applicativo in pacchetti a livello di trasporto, noti secondo il gergo come **segmenti** (transport-lager segment)

  La creazione di segmenti di base consiste nell'aggiungere a un messaggio un'intestazione di trasporto, se il messaggio però fosse troppo grande, questo verrebbe spezzettato e ad ogni sua pezzetto verrà riservato lo stesso trattamento

  A questo punto triplo di trasporto passa il segmento livello di rete, tu hai viene incapsulato l'interno di un pacchetto a livello di rete (data gramma) e inviato a destinazione

  Durante il tragitto i router intermedi, che faranno "rimbalzare i datagrammi", agiranno sono sui campi livello di rete del datagramma, senza esaminare i campi del segmento incapsulato al suo interno

- e dal lato ricevente

  Il livello di rete estrae il segmento dal datagramma e lo passa al livello superiore, cioè a quello di trasporto

  Il livello di trasporto allora elaborerà il segmento ricevuto rendendo disponibili all'applicazione destinataria i dati del segmento

E ricordiamo che le applicazioni di rete vengono forniti più di un protocollo libero di trasporto, tra cui ricordiamo TCP e UDP

3.1.1 - Relazione tra i livelli di trasporto e di rete

ES - analogia posta
messaggi dell'applicazione =
= lettere nelle buste
processi = coloro che ricevono
la posta
host (sistemi periferici) = condomini
protocollo a livello di trasporto =
= chi raccoglie e distribuisce la posta
protocollo a livello di rete =
= servizio postale

Quindi, ricapitolando

- un protocollo a livello di trasporto
  mette a disposizione una comunicazione logica tra processi che vengono eseguiti su host diversi
- un protocollo a livello di rete
  fornisce comunicazione logica tra host

3.1.2 - Panoramica del livello di trasporto di internet

Abbiamo già citato che Internet mette a disposizione del livello di applicazione due diversi protocolli:

- **UDP** (User Datagram Protocol) che è:
  - NON affidabile e
  - NON orientato alla connessione
- **TCP** (Transmission Control Protocol) che è:
  - affidabile e
  - orientato alla connessione

Oltre a questi dobbiamo accennare un protocollo a livello di rete, ovvero

- **IP** (Internet Protocol)
  Esso fornisce:
  - comunicazione logica tra host e
  - un modello di servizio detto **best-effort delivery service** o **best-effort** (massimo sforzo)
    Ovvero fa del suo meglio per far arrivare a destinazione i segmenti inviati, ma non offre alcuna garanzia in merito, né in termini di consegna, né in termini di integrità, pertanto il servizio possiamo categorizzarlo come non affidabile

TCP e UDP estendono il servizio di consegna di IP "tra sistemi periferici" a quello di consegna "tra processi in esecuzione sui sistemi periferici", questo passaggio da

- consegna host-to-host a]
- consegna process-to-process]

multiplexing e **demultiplexing a livello di trasporto** (transport layer multiplexing/demultiplexing) che vedremo a breve

Oltre ai tipi di servizi offerti da UDP e TCP c'è ne sono altri tra cui:

- controllo di integrità (sia per UDP che per TCP) attraverso l'inclusione di campi per il riconoscimento di errori nelle intestazioni dei propri segmenti

- **controllo di congestione** (congestion control) (solo per TCP)

  che consiste nell'evitare che le connessioni TCP intasino i collegamenti e i router tra gli host comunicanti con un'eccessiva quantità di traffico o che tantomeno in un collegamento congestionato le connessioni condividano equamente la larghezza di banda del collegamento stesso, il tutto regolando la velocità (rate) alla quale il lato mittente della connessione TCP immette traffico in rete

3.2 - Multiplexing e demultiplexing

Ripasso

TCP e UDP estendono il servizio di consegna di IP "tra sistemi periferici" a quello di consegna "tra processi in esecuzione sui sistemi periferici", questo passaggio da

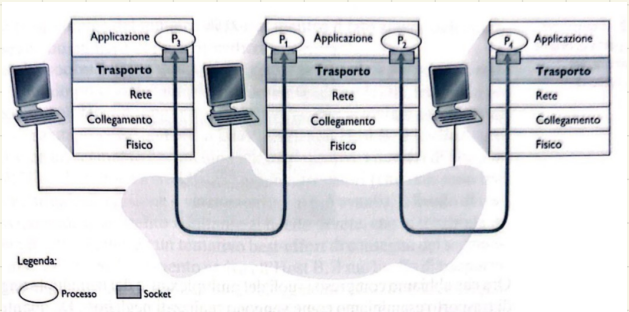
- consegna host-to-host a
- consegna process-to-process

] **multiplexing** e **demultiplexing a livello di trasporto** (transport layer multiplexing/demultiplexing)

In merito al ripasso, è bene specificare che un processo può gestire una o più socket, attraverso le quali i dati fluiscono dalla rete al processo e viceversa

Di conseguenza il livello di trasporto nell'host di ricezione in realtà non trasferisci dati direttamente a un processo, ma piuttosto a una socket che fa da intermediario

E siccome, a ogni dato istante, può esserci più di una socket nell'host di ricezione, ciascuna avrà un identificatore univoco il cui formato dipende dal fatto che si tratti di socket UDP o TCP



Quindi ciascun segmento di valori trasporta vari campi deputati allo scopo ed è andato ricevente il libro di trasporto esamina questi campi per identificare la socket di ricezione a cui poi vi dirigerà il segmento

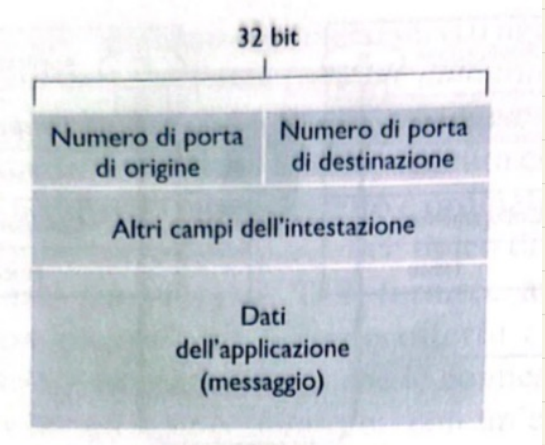
- Il compito di trasportare i dati dei segmenti a livello di trasporto verso la giusta socket viene detto **demultiplexing**

- Il compito di radunare frammenti di dati da diverse socket sull'host di origine e incapsulare ognuno con intestazioni di a livello di trasporto per creare dei segmenti e passarli a livello di rete, viene detto **multiplexing**

  Guardando l'immagine si può osservare che il livello di trasporto nell'host centrale deve effettuare il demultiplexing dal livello di rete di segmenti che possono arrivare sia per il processo P1, sia per P2, indirizzando i dati del segmento in ingresso alla giusta socket

Compresi i ruoli, ora focalizziamoci nel comprendere come vengono realizzati negli host

Abbiamo visto che il Multiplexing a livello di trasporto richiede
1. che le socket abbiano identificatori unici e
2. che ciascun segmento presenti campi che indichino la socket cui va consegnato il segmento



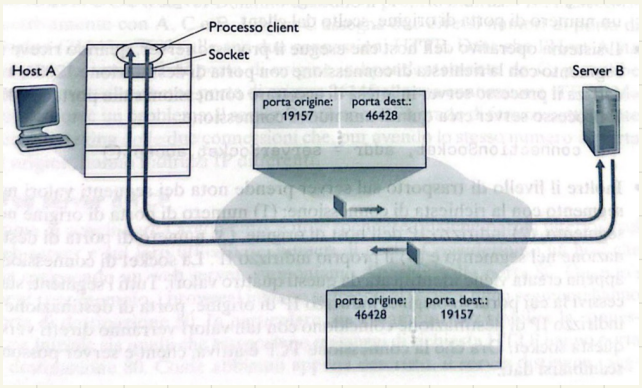
Questi sono:
1. il **campo del numero di porta di origine**
2. (source port Numbers field) e il **campo del numero di porta di destinazione** (destination port Numbers field)

I numeri di porta sono di 16 bit e vanno da zero a 65.535, tuttavia quelli che vanno da zero a 1023 sono chiamati **numeri di porta noti** (well-known port Numbers) e sono riservati per essere usati dai protocolli ben noti (HTTP - porta 80)

Siccome però i segmenti possono essere TCP e UDP il formato appena visto può cambiare così anche l'approccio di demultiplexing a livello di trasporto a seconda del tipo di segmento

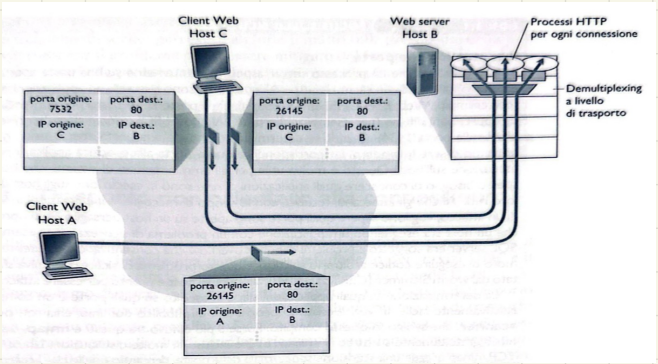
Molto velocemente:

- se il segmento è UDP, il formato del segmento è quello appena visto e il demultiplexing è fatto solamente basandosi sul numero di porta di destinazione



Pertanto, considerando che una socket UDP viene identificata completamente da una coppia che consiste di un indirizzo IP e di un numero di porti destinazione, due segmenti UDP che presentano diversi numeri di porta di origine, ma hanno stesso indirizzo numero di porta di destinazione, vengono diretti allo stesso processo di destinazione tramite la medesima socket

- se il segmento è TCP, il formato del segmento presenta, oltre alla coppia numeri di porta, un'altra coppia di indirizzi IP, di origine e di destinazione e il demultiplexing viene fatto basandosi su tutti e 4 i valori



Pertanto, considerando che una socket TCP viene identificata completamente da due coppie, una che consiste di un indirizzo IP di ordine e di destinazione e una che consiste di un numero di porta di origine e di destinazione, due segmenti TCP che presentano diversi numeri di porta di origine, ma hanno gli stessi valori restanti, vengono diretti a diversi processi tramite diverse socket

3.3 - Trasporto non orientato alla connessione: UDP

Rifacendoci a quanto ho detto precedentemente e tenendo conto che il livello di trasporto deve fornire un servizio di multiplexing/demultiplexing, al fine di trasferire dati tra il livello di rete e il processo corretto al livello di applicazione, UDP fa praticamente il minimo che è un protocollo di trasporto debba fare, tanto è vero che abbiamo visto che esso non aggiunge nulla a IP

La volontà da parte dello sviluppatore di scegliere UDP rispetto a TCP ricade sul fatto che l'applicazione dialoga quasi modo diretto con IP tanto è vero che UDP:

- prende l'ID messaggi dal processo applicativo
- aggiunge il numero di porte di origine di destinazione per il multiplexing/demultiplexing
- aggiunge altri due piccoli campi e
- passa il segmento risultante al livello di rete

A quel punto il livello di rete incapsula il segmento in un datagramma IP e quindi effettua un tentativo di consegnarlo all'host di destinazione in modalità best-effort

Se il segmento arrivo a destinazione, UDP utilizza il numero di porte di destinazione per consegnare i dati del segmento al processo applicativo corretto

ES - DNS è un tipico esempio di protocollo a livello applicativo che utilizza UDP, quando viene creato il messaggio di query, questo viene passato a UDP

In tutta questa procedura, rispetto a quanto visto in TCP, in UDP non c'è l'handshaking tra l'entità di invio di ricezione a livello di trasporto, tanto è vero che abbiamo categorizzato UDP come *non orientato alla connessione* (connectionless)

Vediamo nello specifico alcuni motivi per scegliere UDP

- **Controllo più preciso a livello di applicazione su quali dati sono inviati e quando**
  Non appena un processo applicativo passa dei dati a UDP, quest'ultimo li impacchetta in un segmento che trasferisce immediatamente al livello di rete

  TCP invece, disponendo di sistemi di controllo della congestione e verifica di avvenuta consegna (ACK) fa ritardare di molto la consegna completa di dati

- **Nessuna connessione stabilita**
  UDP non svolge nessun handshake e non introduce alcun ritardo nello stabilire una connessione, manda i segmenti incurante dell'avvenuta consegna

  TCP invece svolge handshake a tre vie e verifica che la connessione stata stabilita prima di inviare

- **Nessuno stato di connessione**
  UDP non conserva lo stato della connessione e non tiene traccia di parametri come:
  - buffer di ricezione e di invio
  - parametri per il controllo di congestione e
  - parametri sul numero di sequenza
  - parametri sul numero di acknowledgment

TCP invece fa completamente l'opposto di quanto scritto al fine di implementare i servizi di trasferimento dati affidabile e per fornire il controllo di congestione

- **Minor spazio usato per l'intestazione del pacchetto**
  L'intestazione dei pacchetti UDP aggiunge 8 bye

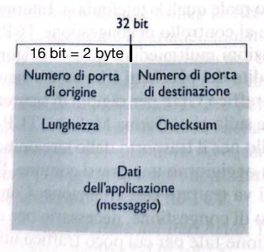
TCP 20 byte

Riportiamo nuovamente una tabella che illustra quali servizi fanno uso di quella protocollo di trasporto

Applicazione	Protocollo a livello di applicazione	Protocollo di trasporto sottostante
Posta elettronica	SMTP	TCP
Accesso a terminali remoti	Telnet	TCP
Accesso sicuro a terminali remoti	SSH	TCP
Web	HTTP, HTTP/3	TCP (per HTTP), UDP (per HTTP/3)
Trasferimento file	FTP	TCP
Server di file remoti	NFS	generalmente UDP
Dati multimediali in streaming	DASH	TCP
Telefonia su Internet	generalmente proprietario	UDP o TCP
Gestione di rete	SNMP	generalmente UDP
Traduzione di nomi	DNS	generalmente UDP

3.3.1 - Struttura dei segmenti UDP

L'intestazione UDP presente solo quattro campi di 2 byte ciascuno:
- Numero di porta di origine
- Numero di porta di destinazione
- Lunghezza e
- Checksum



I numeri di porta abbiamo già visto e detto che consentono all'host di destinazione di trasferire i dati applicativi al processo corretto (demultiplexing)

Il campo "Lunghezza" specifica il numero di base del segmento UDP ed è necessario perché la grandezza del campo "Dati" può essere diverso tra un segmento e quello successivo

Il campo "Checksum" viene usato dall'host ricevente per verificare se sono avvenuti errori nel segmento

3.3.2 - Checksum UDP

Come già anticipato, il Checksum UDP serve per il rilevamento degli errori, ovvero viene utilizzato per determinare se i bit del segmento UDP sono stati alterati durante il loro trasferimento la sorgente a destinazione

ES -

0110011001100000
0101010101010101
1000111100001100

La somma delle prime due è:

0110011001100000
0101010101010101
1011101110110101

Sommando la terza parola al risultato precedente otteniamo:

1011101110110101
1000111100001100
0100101011000010

111111111111
1011010100111101 = Checksum

somma al primo bit

Per verificare ciò l'host destinatario effettua il completamento a 1 della somma di tutte le parole da 16 bit nel segmento e l'eventuale riporto finale viene sommato al primo bit, il risultato ottenuto viene poi messo nel campo Checksum del segmento UDP

- Se non ci sono errori nel pacchetto, l'addizione darà 1111111111111
- altrimenti se un bit vale 0 sappiamo che è stato introdotto un errore

Nonostante UDP sia categorizzato come affidabile e molti protocolli a livello di collegamento prevedono il controllo degli errori, il Checksum è messo a disposizione perché non c'è garanzia che tutti i collegamenti tra origine destinazione controlli non gli errori

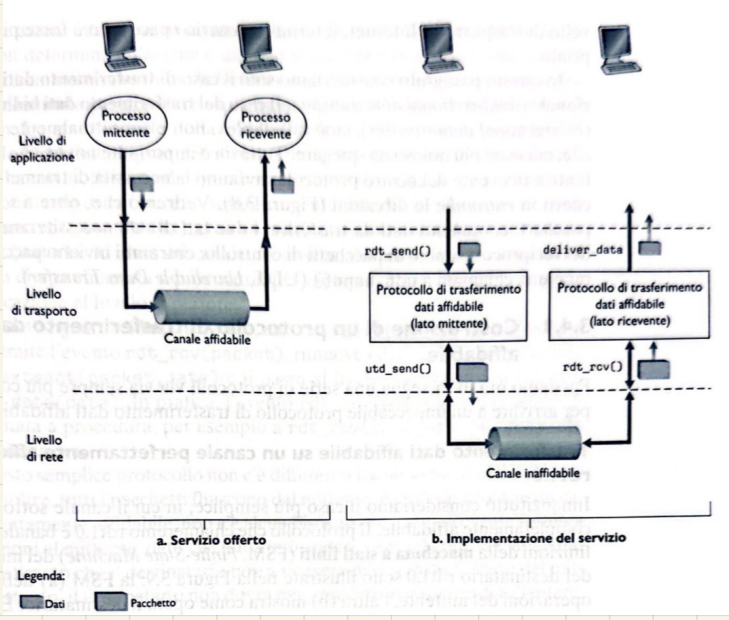
In aggiunta, se i segmenti fossero trasferiti correttamente lungo un collegamento, si potrebbe verificare un errore mentre il segmento si trova nella memoria di un router

ES principio end-to-end nella progettazione di sistemi
In base al quale, dato che determinate funzionalità devono essere implementate su base end-to-end, le funzionalità posizionate ai livelli inferiori possono diventare ridondanti o di scarso valore se confrontate con le stesse funzionalità offerte ai livelli superiori

3.4 - Principi del trasferimento dati affidabile

Adesso andiamo a trattare il problema del trasferimento dati affidabile in un contesto generale

Innanzitutto è bene precisare che il trasferimento dati affidabile si verifica non solo a livello di trasporto, ma anche a livello di rete e di applicazione



L'astrazione del servizio offerto all'entità dei livelli superiori è quella di un canale affidabile tramite il quale si possono trasferire i dati, dati che, viaggiando attraverso un canale affidabile, nessuno dei loro bit rimane corrotto o va perduto e vengono tutti consegnati nell'ordine di invio

Il compito di un **protocollo di trasferimento dati affidabile** (replicabile data transfer protocol)e l'implementazione di questa astrazione del servizio

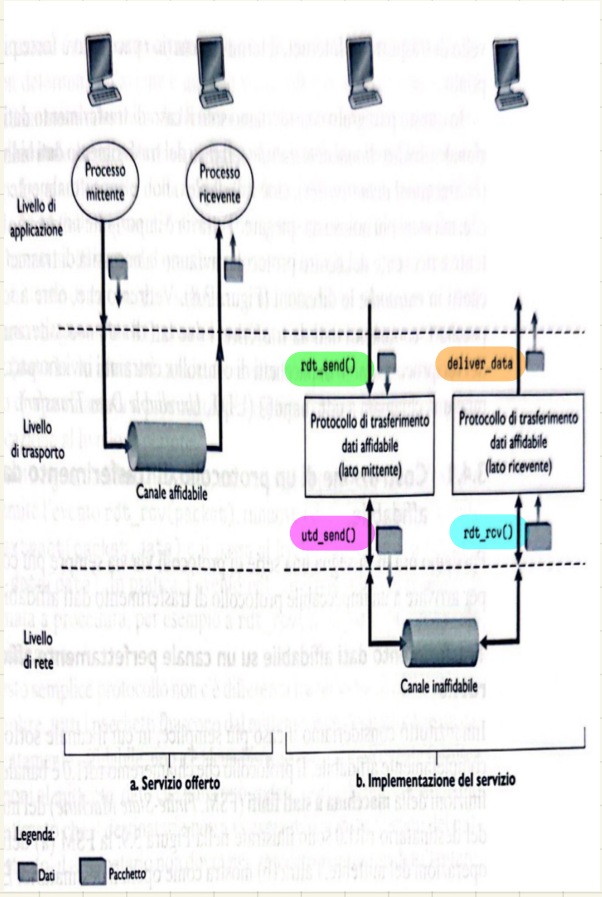
Ciò è complicato però dalla possibilità inafidabilità del livello “al di sotto” del protocollo di trasferimento dati

La situazione generale in cui ci porremo per comprendere il funzionamento dietro è la seguente:

Assumeremo che dei pacchetti vengano consegnati nell’ordine con cui sono stati inviati, ma alcuni si andranno a perdere o anche che alcuni andranno persi, ovvero il canale sottostante non riordina i pacchetti

Guardando l’immagine vediamo che il lato mittente del protocollo trasferimento dati sarà invocato tramite una chiamata a “rdt_send( )” e trasferire i dati da consegnare a livello superiore sul lato ricevente, dove:

- “rdt”, in questo caso, sta per “reliable data transfer” (trasferimento dati affidabile)
- “_send” indica la chiamata al lato mittente di “rdt”



Prima che un pacchetto abbia raggiunto il lato ricevente, i due lati di rdt necessiteranno anche del reciproco scambio di pacchetti di controllo inviandosi pacchetti tramite una chiamata a “udt_send( )” (unreliable data transfer)

Quando un pacchetto raggiunge il lato ricevente del canale, verrà chiamata “rdt_rcv( )”

Nel momento in cui il protocollo rdt voglia consegnare i dati al livello superiore, lo farà chiamando “deliver_data( )”

Di tutto questo discorso considereremo solo il caso di **trasferimento dati unidirezionale** (unidiretional data transfer)

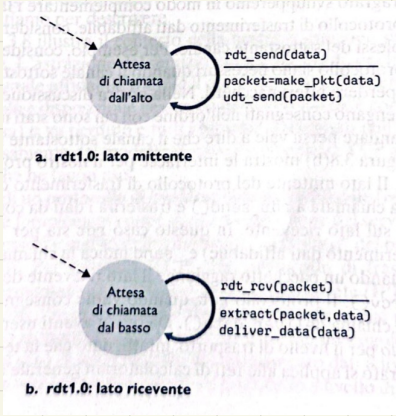
3.4.1 - Costruzione di un protocollo di trasferimento dati affidabile

- **Trasferimento dati affidabili su un canale perfettamente affidabile: rdt 1.0**

  Il caso più semplice in cui il canale sottostante è completamente affidabile

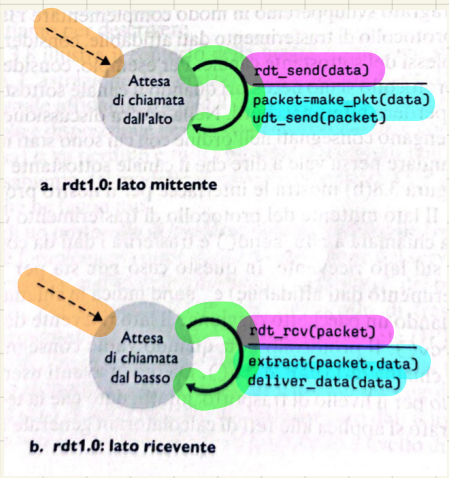
  Il protocollo, che chiameremo rdt1.0 è alquanto semplice

  Le definizioni di **macchina a stati finiti** (FSM, Finite-State Machine) del mittente e del destinatario sono quelli illustrate



Dalle immagini vediamo che le FSM:

- sono separate e
- hanno un unico stato ovvero le transizioni (indicate dalle frecce) hanno luogo necessariamente tra quello stato e sé stesso



Lo stato iniziale della FSM è indicato dalla freccia tratteggiata

Si osservi anche che:

- l'evento che causa la transizione è scritto sopra la linea orizzontale che la etichetta, mentre
- le azioni intraprese in seguito all'evento sono scritte sotto

Per capire la simbologia:

- quando un evento non determina un'azione e
- quando viene intrapresa un'azione senza il verificarsi di un evento

] useremo a lettera greca Λ rispettivamente

- sotto o
- sopra

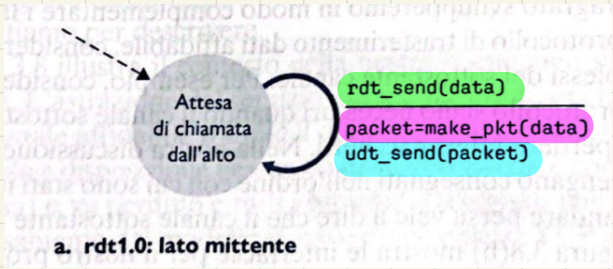
la linea orizzontale per denotare esplicitamente la mancanza di un'azione o di un evento

Andiamo adesso ad esaminare i lati

- **lato mittente**

  Il lato mittente rdt:

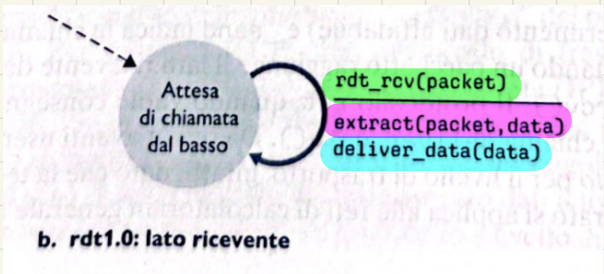
  - accetta dati dal livello superiore tramite l'evento "rdt_send(data)"
  - crea un pacchetto contenente dati con l'azione "make_pkt(data)" e
  - invia il pacchetto con l'azione "udt_send(packet)"



- **lato ricevente**

  Il lato ricevente rdt:

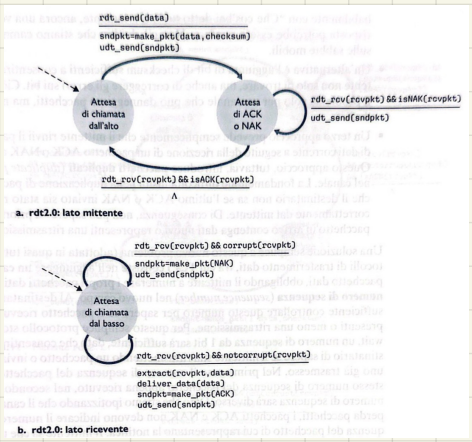
  - raccoglie i pacchetti dal canale sottostante tramite l'evento "rdt_rcv(packet)"
  - rimuove i dati dei pacchetti tramite l'azione "extract(packet, data)" e
  - li passa al livello superiore con l'azione "deliver_data(data)"



Con un canale perfettamente affidabile, non c'è alcun bisogno che il dato ricevente fornisca informazioni al mittente dato che nulla può andare storto

- **Trasferimento dati affidabili su un canale con errori sui bit: rdt 2.0**

Un caso più realistico del canale sottostante è quello in cui i bit possono essere corrotti



Tali errori si verificano nei componenti fisici delle reti quando il pacchetto viene:

- trasmesso
- propagato o
- inserito nei buffer

Nonostante la situazione, continueremo ad assumere che tutti i pacchetti trasmessi vengono ricevuti nell'ordine di invio

Per risolvere tale problema rinvieremo i messaggi corrotti
Per sapere se il messaggio è corrotto si userà un protocollo di dettatura dei messaggi che usa **notifiche (acknowledgment) positive** e **negative**

Tali messaggi di controllo consentono al destinatario di far sapere al mittente che cosa sia stato ricevuto correttamente e che cosa no, chiedendone quindi la ripetizione

Nel contesto di una rete di calcolatori, i protocolli di trasferimento dati affidabili basati su ritrasmissioni sono noti come **protocolli ARQ** (Automatic Repeat Request)

Fondamentalmente, per gestire la presenza di errori nei bit, i protocolli ARQ devono avere tre funzionalità aggiuntive:

1. **Rilevamento dell'errore** (error detection)
   È richiesto al destinatario un meccanismo che consente di rilevare gli errori sui bit

2. **Feedback del destinatario** (receiver feedback)
   Nonostante il mittente e destinatario operino su sistemi periferici diversi, distanti migliaia di chilometri, l'unico modo che il mittente ha per avere la prospettiva del destinatario è quella di avere un feedback esplicito con quest'ultimo

   Per fare ciò il protocollo rdt 2.0 manderà pacchetti:
   - ACK (notifica positiva) e
   - NACK (notifica negativa)

   dal destinatario al mittenet

3. **Ritrasmissione** (retransmission)
   Un pacchetto ricevuto con errori sarà ritrasmesso dal mittente

Di seguito un illustrazione

Ripasso:

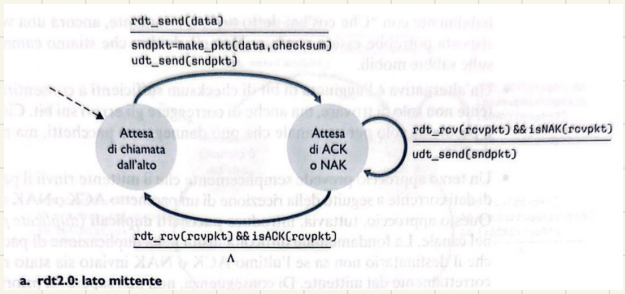
- stato (freccia)
- evento che causa la transizione (sopra linea orizzontale)
- azioni intraprese in seguito all'evento (sotto linea orizzontale)
- lettera greca Λ per denotare esplicitamente la mancanza di un'azione o di un evento

Andiamo adesso ad esaminare i lati

- **lato mittente**

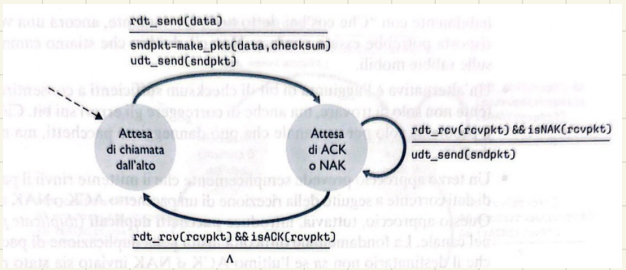
  Il lato mittente rdt presenta due stati:

  1. Nello stato di sinistra, il protocollo lato mittente sta attendendo i dati da raccogliere da livello superiore
     - quando si verifica l'evento "rdt_send(data)"
     - il mittente crea un pacchetto (sndpkt) con l'azione "make_pkt(data, checksum)" contenente i dati da inviare e il checksum e
     - infine spedisce il pacchetto tramite l'operazione "udt_send(sndpkt)"



a. rdt2.0: lato mittente

  2. Nello stato di destra, il protocollo mittente in attesa di un pacchetto ACK o NACK dal destinatario
     - se riceve un ACK, evento denotato da rdt_rcv(rcvpkt) && isACK(rcvpkt), il mittente sa che il pacchetto trasmesso più di recente è stato ricevuto correttamente, pertanto il protocollo ritorna allo stato di attesa dei dati provenienti dal livello superiore
     - se invece riceve un NACK, il protocollo ritrasmette l'ultimo pacchetto e attendo una risposta dalla trasmissione

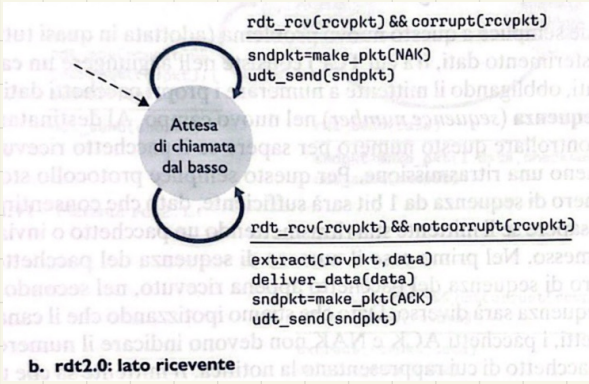


È importante notare che quando il mittente nello stadio attesa di ACK o NACK, non può recepire i dati dal livello superiore, in pratica non può avere luogo l'evento "rdt_send( )" evento che si verificherà solo dopo la ricezione di un ACK che permette al mittente di cambiare stato

Il mittente quindi non invia nuovi dati finché non è certo che destinatario abbia ricevuto correttamente il pacchetto corrente, questo comportamento ha portato a rendere i protocolli rdt 2.0 noti come protocollo **stop-and-wait**

- **lato ricevente**
  Il lato ricevente rdt presente un solo stato

  All'arrivo del pacchetto, il destinatario risponde o con ACK un o con un NACK, a seconda che il pacchetto sia corrotto meno



b. rdt2.0: lato ricevente

  Se pur all'apparenza sembri che il protocollo rdt 2.0 funzioni, è presente un grave difetto, ovvero che i pacchetti ACK e NACK potrebbero essere a alterati, in tal caso ci sarebbero due scenari possibili:

  1. Aggiunta di un bit di Checksum sufficiente a consentire al mittente non so ritrovare, ma anche di correggere gli errori sui bit

     Così facendo si risolverebbe il problema a patto che il canale di trasmissione si limiti solo a danneggiare i pacchetti e non a perderli

  2. Rinvio del pacchetto di dati corrente a seguito della ricezione di una ACK o NACK alterato

     Così facendo si introdurrebbe un nuovo problema che è quello dei **pacchetti duplicati** nel canale

     Il problema principale con la duplicazione di pacchetti è che il destinatario non sa se l'ultimo ACK o NACK inviato sia stato ricevuto correttamente dal mittente, pertanto non può sapere a priori se un pacchetto in arrivo contenga dati nuovi o rappresenti una ritrasmissione

  Una soluzione semplice a questo nuovo problema consiste nell'aggiungere un campo al pacchetto dati, obbligando il mittente a numerare i propri pacchetti dati con un **numero di sequenza** nel nuovo campo

  Così facendo al destinatario sarà sufficiente controllare questo numero per sapere se il pacchetto ricevuto rappresenti o meno 1 ritrasmissione

  Per questo semplice protocollo stop-and-wait un numero di sequenze da 1 bit sarà sufficiente, dato che consentirà al destinatario di sapere se il mittente stia ritrasmettendo un pacchetto o inviandone uno già trasmesso

  E siccome ci eravamo messi nell'ipotesi in cui il canale non perdesse pacchetti, i pacchetti ACK e NACK non devono indicare il numero di sequenze del pacchetto di cui rappresentano la notifica, poiché il mittente sa che un pacchetto ricevuto di tipo ACK o NACK (alterato o meno) è stato generato come risposta al pacchetto dati trasmesso più di recente

Con tutte queste nuove modifiche le SFM di mittente e destinatario hanno ora il doppio degli stati precedenti

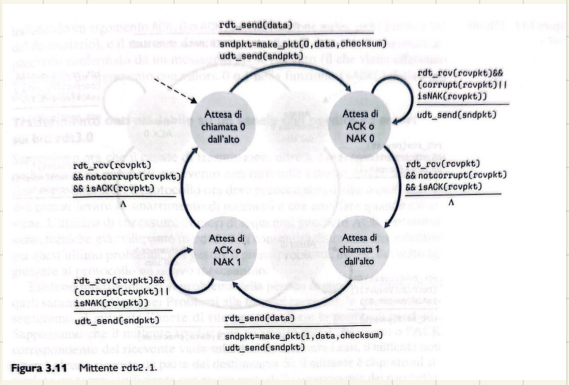


Figura 3.11 Mittente rdt2.1.

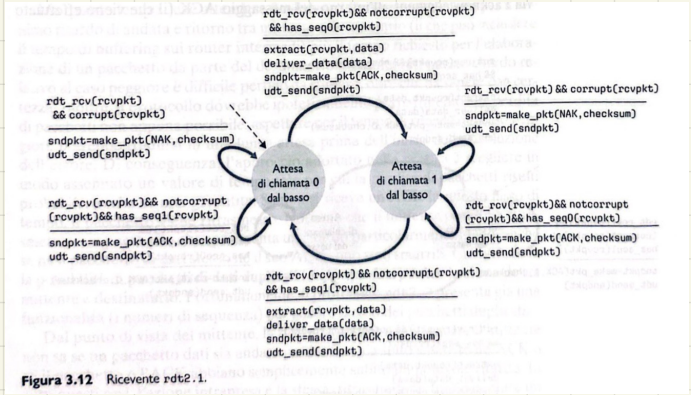


Figura 3.12 Ricevente rdt2.1.

Questo avviene perché lo stato del protocollo deve riflettere il fatto che il pacchetto attualmente in invio o in ricezione abbia numero di sequenza 0 o 1

Notiamo inoltre che le azioni negli stati di invio e di attesa di un pacchetto numerato 0, sono immagini speculari delle azioni negli stati in cui viene spedito o si attende un pacchetto numero 1

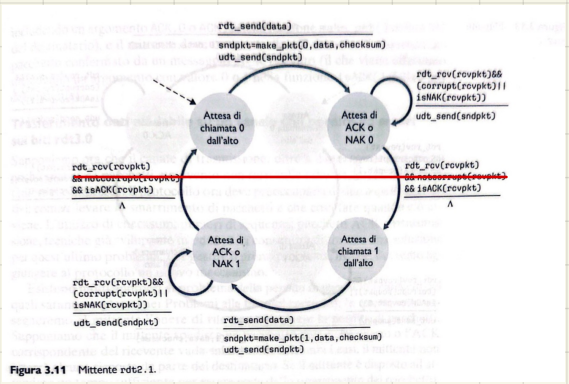


Figura 3.11 Mittente rdt2.1.

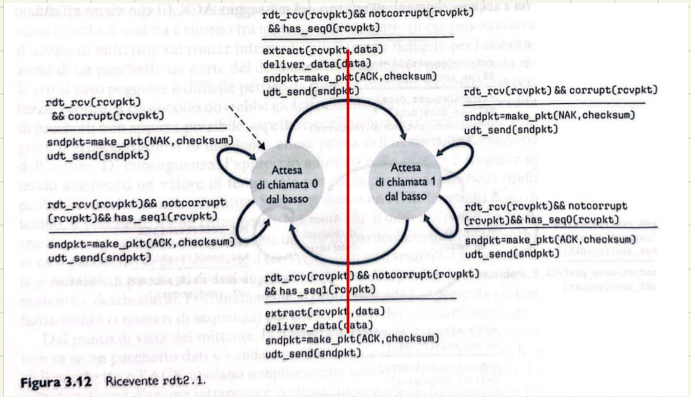


Figura 3.12 Ricevente rdt2.1.

L'unica differenza riguarda la gestione del numero di sequenza

Il protocollo rdt 2.1 usa ACK positivi e negativi del destinatario verso il mittente, destinatario che manda

- un ACK positivo quando ricevo un pacchetto fuori sequenza
- un ACK negativi quando riceve un pacchetto alterato

Possiamo ottenere lo stesso effetto di un NAK spendendo piuttosto un ACK per il più recente pacchetto ricevuto correttamente

Un mittente che riceve due ACK per lo stesso pacchetto (ossia riceve **ACK duplicati**) so che il destinatario non ha ricevuto correttamente il pacchetto successivo a quello confermato due volte

Il nostro protocollo di trasferimento dati affidabile e privo di NACK per un canale con errori sui bit è **rdt 2.2**, che rispetto a rdt 2.1:

- il destinatario deve ora includere il numero di seguenze del pacchetto di cui invia l'ACK all'interno del messaggio ACK (avviene includendo un argomento ACK,0 o ACK,1 nella funzione "make_pkt( ) della SFM del destinatario) e

- il mittente e deve ora controllare il numero di seguenze del pacchetto confermato da un messaggio ACK ricevuto (avviene includendo un argomento con valore 0 i 1 nella funzione isACK( ) della SFM del mittente)

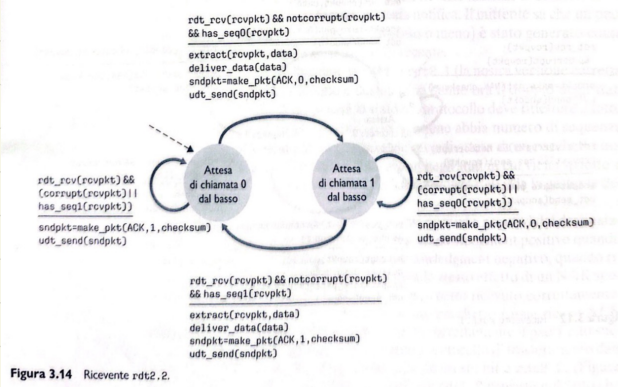
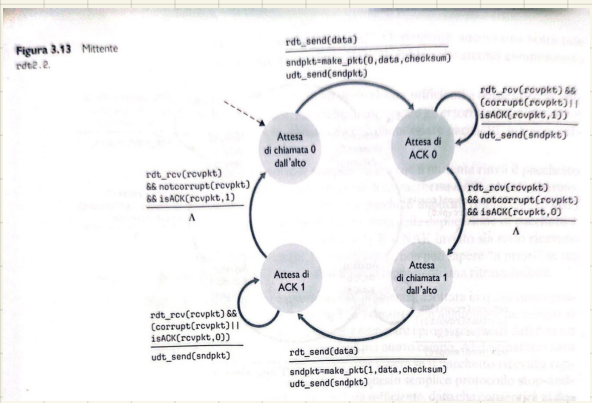


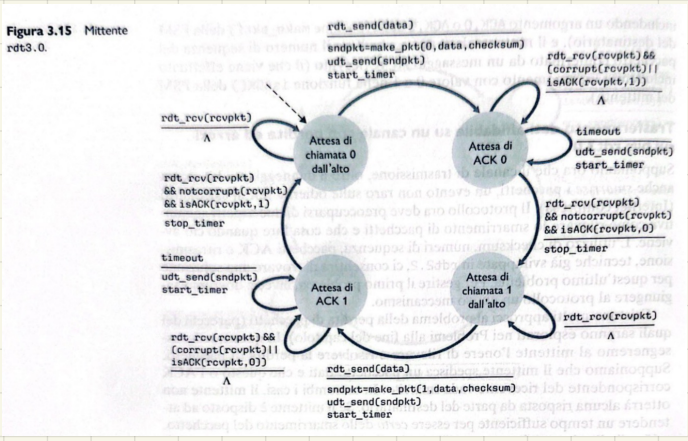
Figura 3.14 Ricevente rdt2.2.



- **Trasferimento dati affidabili su un canale con perdite ed errori sui bit: rdt 3.0**

Supponiamo ora che il canale di trasmissione, oltre a danneggiare i bit, possa anche smarrire i pacchetti , pertanto ora il protocollo dovrà occuparsi di due aspetti aggiuntivi

1. come rilevare lo smarrimento di pacchetti e
2. che cosa fare quando ciò avviene



Per gestire il problema (1) dovremmo aggiungere al protocollo un nuovo meccanismo

Supponiamo che il mittente spedisco un pacchetto dati e che questo o l'ACK corrispondente del ricevente vada smarrito

In entrambi i casi, il mittente non otterrà alcuna risposta da parte del destinatario

Se il mittente è disposto ad attendere un tempo sufficiente per essere certo dello smarrimento del pacchetto, può semplicemente di trasmettere il pacchetto di dati e questi tempo sarà pari a:

- il minimo ritardo di andata e ritorno tra mittente e destinatario +
- il tempo richiesto per l'elaborazione di un pacchetto da parte del destinatario

Inoltre, il protocollo dovrebbe ipoteticamente porre rimedio alla perdita dei pacchetti non appena possibile

Tuttavia, aspettare per il tempo di ritardo nel caso peggiore potrebbe tradursi in una lunga attesa prima dell'inizio della risoluzione dell'errore

Di conseguenza, l'approccio adottato nella pratica è scegliere in modo saggio un valore di tempo tale per cui la perdita di pacchetti risulta probabile, anche se non garantita

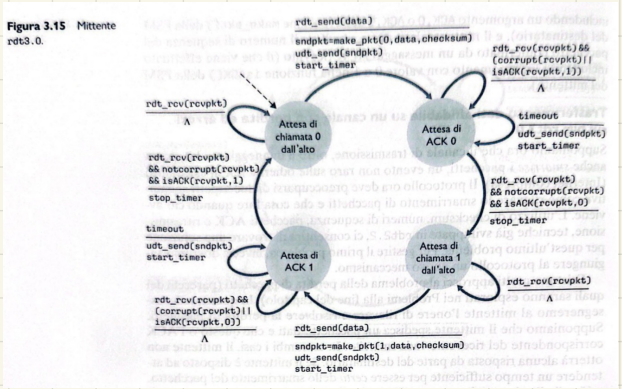
- Se non si riceve un ACK in questo lasso di tempo, il pacchetto venerdì trasmesso
- Se un pacchetto spedito ha un tempo di ritardo particolarmente lungo, più del tempo di attesa, esso viene ritrasmesso anche se non è andato smarrito, introducendo quindi la possibilità di **pacchetti dati duplicati** sul canale tra mittente e destinatario

Dal punto di vista del mittente, l'hai trasmissione è una soluzione universale, il mittente non sa:

- se un pacchetto dati sia andato perduto
- se sia stato smarrito un ACK o
- se il pacchetto o l'ACK abbiano semplicemente subito un notevole ritardo

in tutti questi casi l'azione intrapresa è la stessa, ritrasmettere

Implementare un meccanismo di trasmissione basato sul tempo richiede un **contatore** in grado di segnalare al mittente l'avvenuta scadenza di un dato lasso di tempo



Il mittente quindi dovrà essere in grado di inizializzare il contatore ogni volta che:

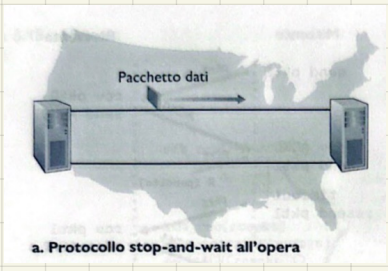
- invia un pacchetto
- risponde a un interrupt generato dal timer con l'azione appropriata e
- di fermare il contatore

3.4.2 - Protocolli per il trasferimento dati affidabile con pipeline

Il protocollo rdt 3.0 ecco corretto dal punto di vista funzionale, tuttavia le prestazioni fanno desiderare, a ragione del fatto che si tratta di un protocollo stop-and-wait

Per valutare l'impatto delle prestazioni, consideriamo il caso ideale di due host

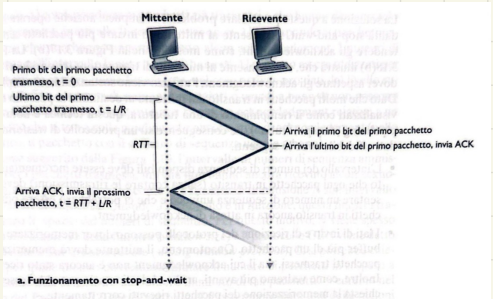
Il ritardo di propagazione di andata e ritorno (RTT) alla velocità della luce per questi due sistemi è approssimativamente di 30 ms



a. Protocollo stop-and-wait all'opera

Supponendo che i due sistemi siano connesse da un canale:

- con velocità di trasmissione R = 1 Gbps e
- con pacchetti di dimensione L di 1000 byte (8000 bit)



a. Funzionamento con stop-and-wait

Il tempo effettivamente richiesto per trasmettere il pacchetto sul collegamento è

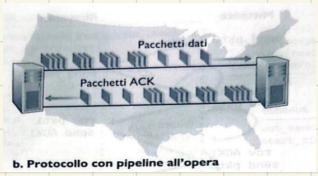
$$d_t = \frac{L}{R} = \frac{8000 \text{ bit}}{10^9 \text{ bit/s}} = 8 \text{ microsecondi}$$

Se invece andiamo a calcolarci il tempo in cui il mittente è stato effettivamente occupato nell'invio di bit sul canale, e qui scopriamo il problema del protocollo stop-and-wait

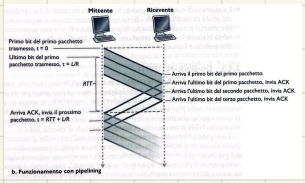
$$U_{mittente} = \frac{L/R}{RTT + L/R} = \frac{0,008}{30,008} = 0,00027$$

Il mittente quindi attivo per solo 2,7 centesimi dell'1% del tempo

Per risolvere questo problema ci basterà andare contro il principio dei protocolli stop-and-wait, ovvero consentire al mittente di mandare più pacchetti senza attendere un ACK



Dato che molti pacchetti in transito al mittente al destinatario possono essere visualizzati come riempimento di una tubatura, questa tecnica è nota come **pipelining**



Facendo così, le conseguenze su un protocollo di trasferimento dati affidabile sono le seguenti:

- l'intervallo di numeri di sequenze disponibili deve essere incrementato, dato che ogni pacchetto in transito (senza contare le trasmissioni) deve presentare un numero di sequenze univoco e che ci potrebbero essere più pacchetti in transito ancora in attesa di ACK
- i dati di invio di ricezione dei protocolli possono dover memorizzare in un buffer più di un pacchetto

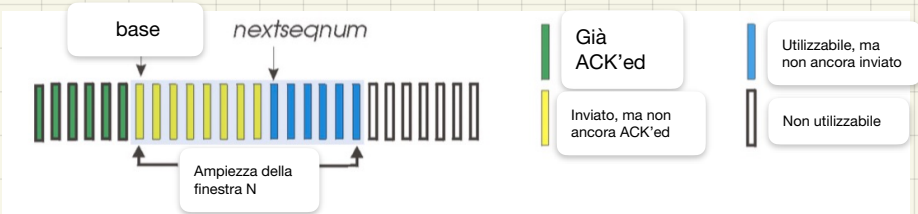
  Quantomeno, il mittente dovrei memorizzare i pacchetti trasmessi, ma il cui ACK non è stato ancora ricevuto

  Inoltre anche il destinatario potrebbe essere richiesta la memorizzazione di pacchetti correttamente
- la quantità di numeri di sequenze necessari e i requisiti di buffer dipendono dal modo in cui i protocolli di trasferimento dati reagisce ai pacchetti smarriti, alterati o troppo in ritardo

  Si possono identificare due approcci di base verso la risoluzione degli errori con pipeline:
  - **Go-Back-N** e
  - **ripetizione selettiva**

3.4.3 - Go-Back-N (GBN)

In un protocollo **Go-Back-N** (GBN) il mittente può trasmettere più pacchetti senza dover attendere alcun ACK, ma può comunque avere più di un dato numero massimo consentito di N pacchetti in attesa di ACK nella pi peline



La figura soprastante ci mostra la visione del mittente sull'intervallo di numeri di sequenza in un protocollo GBN in cui se definiamo:

- come "**base**" il numero di sequenza del pacchetto più vecchio che non ho ancora ricevuto ACK e
- come "**nextseqnum**" il più piccolo numero di sequenza inutilizzato, ovvero il numero di sequenze del prossimo pacchetto da inviare

allora si possono identificare quattro intervalli di numeri di sequenza in cui i numeri di sequenza nell'intervallo:

1. [0, base - 1] corrispondono ai pacchetti già trasmessi e che hanno ricevuto ACK
2. [base, nextseqnum - 1] corrispondono i pacchetti inviati, ma che non hanno ancora ricevuto alcun ACK
3. [nextseqnum, base + N - 1] possono essere utilizzati per i pacchetti da inviare immediatamente
4. [base + N, …] non possono essere utilizzati finché il mittente non riceve un ACK relativo un pacchetto che si trova nella pipeline che è ancora privo di ACK



Come si può vedere l'intervallo di numeri di sequenza ammissibili per i pacchetti trasmessi, ma che non hanno ancora ricevuto alcun ACK, può essere visto come una **finestre dimensione N** sull'intervallo dei numeri di sequenza

Quando il protocollo in funzione, questa finestra trasla lungo lo spazio dei numeri di sequenza, per questa ragione:

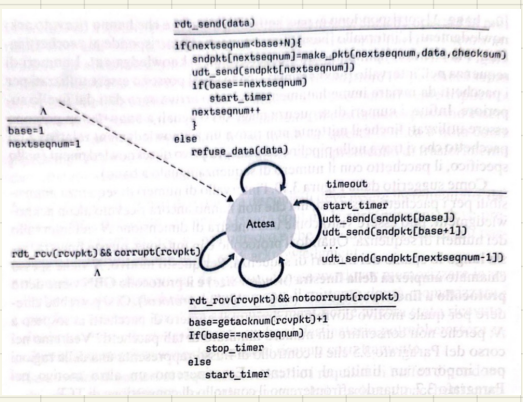
- N viene spesso chiamato ampiezza della finestra (window size) e
- il protocollo GBN viene detto **protocol a finestra scorrevole** (sliding-window protocol)

NOTA - Limite numero di pacchetti e controllo di flusso

La ragione per cui il numero di pacchetti è limitato è per il meccanismo di controllo di flusso, nonché di congestione nello specifico per TCP

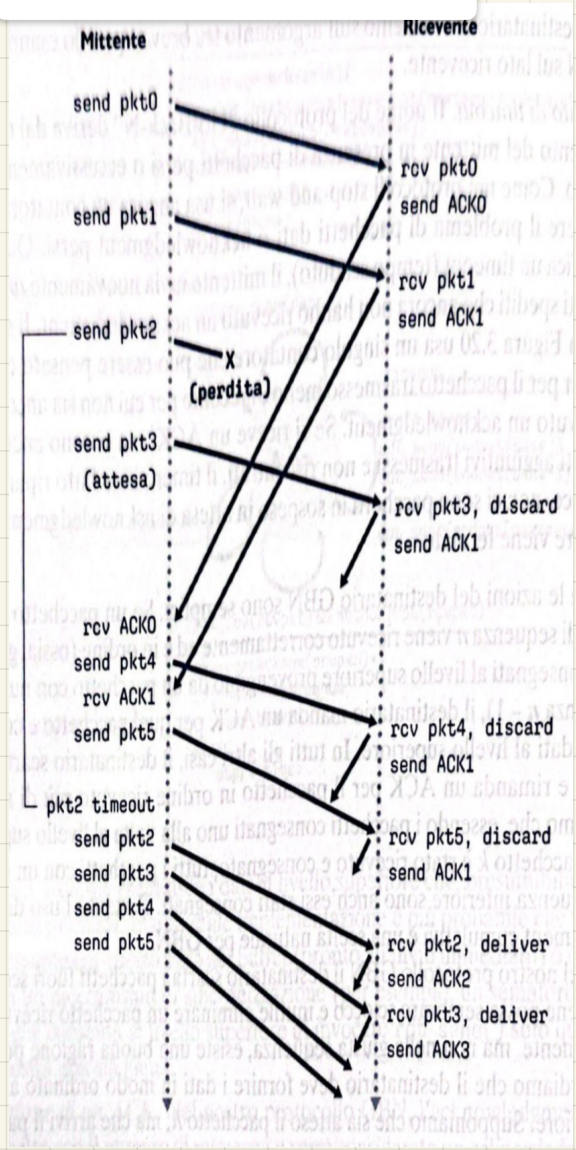
Il mittente GBN deve rispondere a tre tipi di evento:

1. **Invocazione dall'alto**
   Quando dall'alto si chiama "rdt_send( )", per prima cosa il mittente controlla se la finestra sia piena, ossia se vi siano N pacchetti in sospeso senza ACK
   - se una finestra non è piena, crea e invia un pacchetto e le variabili vengono aggiornate di conseguenza



- se la finestra è piena, il mittente restituisce dati a livello superiore che, presumibilmente, ritenterà più tardi

Go-Back-N in funzione



2. **Ricezione di un ACK**
   Nel protocollo GBN, l'ACK del pacchetto con il numero di sequenza n verrà considerato un **ACK cumulativo** (cumulative ACK), che indica che tutti i pacchetti con un numero di sequenza minore uguale n, sono stati ricevuti correttamente dal destinatario

3. **Evento di timeout**
   Il nome del protocollo "Go-Back-N" deriva dal comportamento del mittente in presenza di pacchetti persi o eccessivamente in ritardo

   Come nei protocolli stop-and-wait, si usa ancora un contatore per risolvere il problema di pacchetti dati o ACK persi

   Quando si verifica un "timeout" (tempo scaduto), il mittente invia nuovamente tutti i pacchetti spediti che non hanno ancora ricevuto un ACK

   Quando si riceve un ACK, anche se ci sono ancora pacchetti aggiuntivi trasmessi e non riscontrati, il timer viene fatto ripartire

Anche l'azione del destinatario GBN sono semplici:

- se un pacchetto con numero di sequenza n viene ricevuto correttamente in ordine, allora manda un ACK per quel pacchetto e consegna i suoi dati al livello superiore
- in tutti gli altri casi il destinatario scarta i pacchetti e rimanda un ACK per il pacchetto in ordine ricevuto più di recente

NOTA - ACK cumulativi

Se un pacchetto k viene ricevuto e consegnato al livello superiore, siccome i pacchetti vengono consegnati in ordine, allora tutti gli altri pacchetti con numero di sequenza minore di k sono anch'essi stati consegnati in ordine

NOTA - Ordine VS Memoria

Potrebbe sembrare una scelta obsoleta quella di accettare e consegnare i pacchetti solamente in ordine, dove, nel caso contrario, il pacchetto disordinato verrebbe scartato, tuttavia così facendo il destinatario non deve memorizzare nel buffer i pacchetti che giungono fuori sequenza, con il costo che si potrebbe andare incontro ad ulteriori ritrasmissioni.

Go-Back-N gli = **programmazione basata su eventi** (event-based programming)

3.4.4 - Ripetizione selettiva

Nonostante il protocollo Go-Back-N risolva il problema di scarso utilizzo del canale che abbiamo riscontrato nei protocolli stop-and-wait, allo stesso tempo questo in un diverso scenario può avere problemi di prestazioni

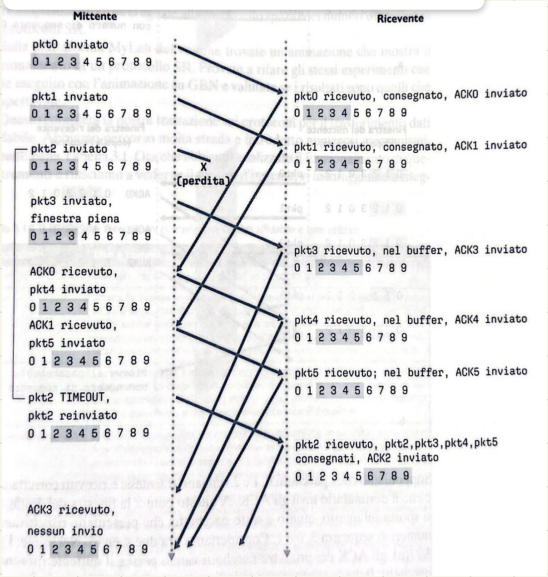
Quando l'ampiezza della finestra e il prodotto tra larghezza di banda in ritardo sono entrambi grandi, nella pipeline si possono trovare numerosi pacchetti e un solo errore su un pacchetto può pertanto provocare un elevato numero di ritrasmissioni, in molti casi inutili e con effetto domino può andare a far saturare la pipeline

In questo scenario però eccellono i **protocolli a ripetizione selettiva** (SR, Selective-Repeat protocol) i quali, dal nome stesso, evitano le ritrasmissioni non necessarie facendo ritrasmettere al mittente solo i pacchetti su cui esistono sospetti di errore

A fronte di ciò il destinatario è costretto a mandare ACK specifici per i pacchetti ricevuti in modo corretto

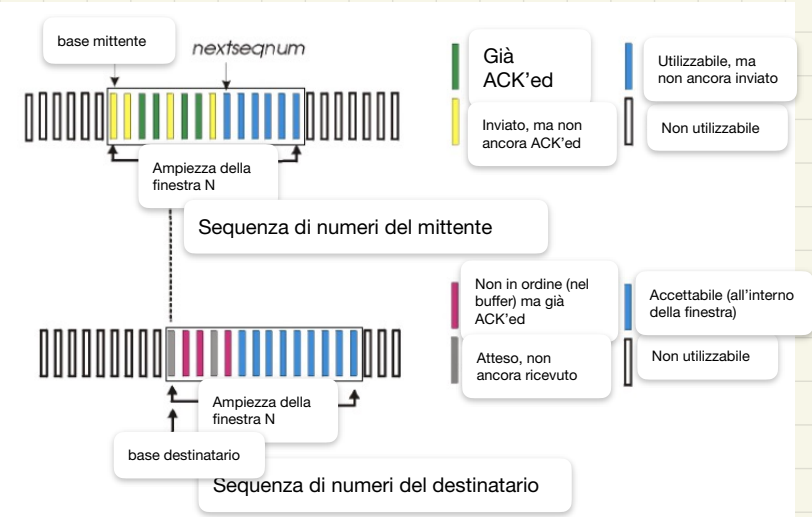
Si userà nuovamente una ampiezza di finestra pari a N per limitare il numero di pacchetti privi di ACK nella pipeline, tuttavia, a differenza di GBN, il mittente avrà già ricevuto gli ACK di qualche pacchetto nella finestra

SR in funzione



Facendo riferimento all'immagine:

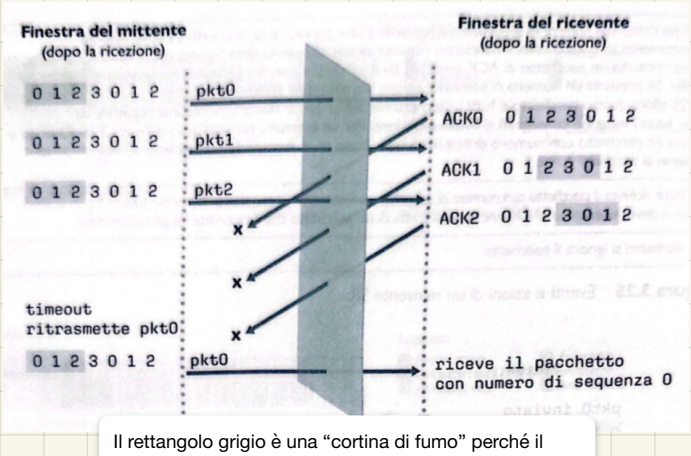
- il destinatario invia un ACK per i pacchetti ricevuti correttamente, che siano in ordine o fuori sequenza e questi vengono memorizzati in un buffer finché non sono stati ricevuti tutti i pacchetti mancanti per completare l'ordine, così da inviarli tutti ordinati al livello superiore
- il destinatario talvolta può optare per rimandare l'ACK di un pacchetto ricevuto per evitare ritrasmissioni
- se il destinatario non invia un ACK di un pacchetto, la finestra del mittente non può avanzare



In merito all'ultimo punto si evidenzia un problema dei protocolli SR, ovvero il mittente e il destinatario non hanno visuale dell'altro e pertanto non sanno cosa sia stato ricevuto correttamente. Difatti le finestre del mittente e del destinatario non sempre coincidono e tale mancanza di sincronizzazione può avere ripercussioni considerando che queste sono di dimensioni finite

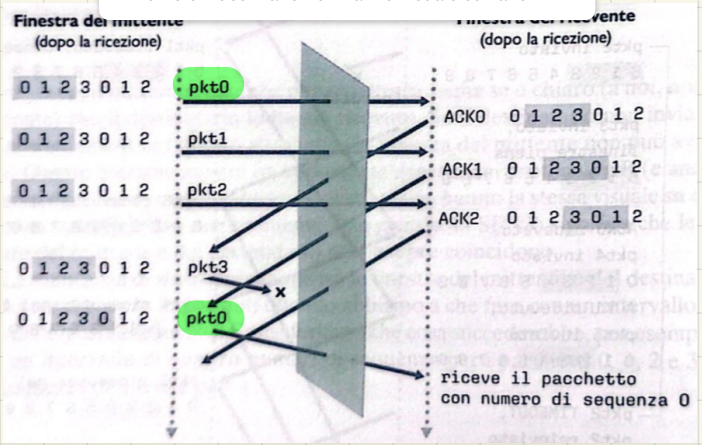
Ci sono due scenari in merito:

1. il mittente ha mandato n pacchetti (pkt0, 1, 2) che sono stati ricevuti e consegnati correttamente dal ricevente il quale ha mandato conseguentemente gli ACK associati (ACK0, 1, 2) che però non sono arrivati a destinazione, quindi se da una parte la finestra di ricezione del destinatario si è spostata, quella del mittente no e al timeout quest'ultimo ha rimandato il primo pacchetto (pkt0) anche se era stato già ricevuto e consegnato



Il rettangolo grigio è una "cortina di fumo" perché il mittente è il destinarlo non hanno visuale sull'alto

2. il mittente ha mandato n pacchetti (pkt0, 1, 2) che sono stati ricevuti e consegnati correttamente dal ricevente il quale ha mandato conseguentemente gli ACK associati (ACK0, 1, 2) che anch'essi sono arrivati correttamente, pertanto entrambe le finestre si spostano e il mittente invia nuovi pacchetti (pkt3, 0) di uno si è perso (pkt3) mentre l'altro è arrivato (pkt0), ma nell'ipotesi che quest'ultimo abbia lo stesso numero di sequenza di un pacchetto già mandato precedentemente, il destinatario non ha idea se questo è un nuovo pacchetto o una ritrasmissione



Di seguito un riepilogo di tutti i meccanismi di trasferimento affidabile e il loro utilizzo

Meccanismo	Uso e commenti
Checksum	Utilizzato per rilevare errori sui bit in un pacchetto trasmesso.
Timer	Serve a far scadere un pacchetto e ritrasmetterlo, forse perché il pacchetto (o il suo ACK) si è smarrito all'interno del canale. I timeout si possono verificare per via dei ritardi anziché degli smarrimenti (timeout prematuro), o quando il pacchetto è stato ricevuto dal destinatario, ma è andato perduto il relativo ACK dal destinatario al mittente. Per questi motivi il destinatario può ricevere copie duplicate di un pacchetto.
Numero di sequenza	Usato per numerare sequenzialmente i pacchetti di dati che fluiscono tra mittente e destinatario. Le discontinuità nei numeri di sequenza di pacchetti ricevuti consentono al destinatario di rilevare i pacchetti persi. I pacchetti con numero di sequenza ripetuto consentono al destinatario di rilevare pacchetti duplicati.
Acknowledgment (ACK)	Utilizzato dal destinatario per comunicare al mittente che un pacchetto o un insieme di pacchetti sono stati ricevuti correttamente. Gli acknowledgement trasporteranno generalmente i numeri di sequenza del pacchetto o dei pacchetti da confermare. A seconda del protocollo, i riscontri possono essere individuali o cumulativi.
Acknowledgment negativo (NAK)	Usato dal destinatario per comunicare al mittente che un pacchetto non è stato ricevuto correttamente. Gli acknowledgement negativi trasporteranno generalmente il numero di sequenza del pacchetto che non è stato ricevuto correttamente.
Finestra e pipelining	Il mittente può essere forzato a inviare soltanto pacchetti con numeri di sequenza che ricadono in un determinato intervallo. Consentendo a più pacchetti di essere trasmessi e non aver ancora ricevuto acknowledgement si può migliorare l'utilizzo del canale rispetto alla modalità operativa stop-and-wait. Vedremo tra breve che l'ampiezza della finestra può essere impostata sulla base della capacità del destinatario di ricevere e memorizzare messaggi in un buffer, su quella del livello di congestione della rete, o su entrambe.

NOTA - Modello VS Realtà

Quando abbiamo parlato del canale di trasmissione, abbiamo posto l'ipotesi che i pacchetti non potessero essere riordinati, un'ipotesi possibile solo se il mittente destinatario sono collegati da un singolo cavo, tuttavia nella realtà gli interlocutori sono collegati attraverso una rete in cui può capitare che i pacchetti vengono mischiati e questa possibilità può portare alla possibile comparsa di vecchie copie di un pacchetto con numero di sequenza o di ACK x anche se non più presente nelle finestre né del mittente né del destinatario. Siccome però i pacchetti devono essere ordinati e si possono riutilizzare i numeri di sequenza, di fronte al problema della duplicazione dei pacchetti, nel riutilizzare un numero di sequenza si presta attenzione nell'assicurarsi che quest'ultimo non sia stato utilizzato per un pacchetto mandato recentemente e che potrebbe ancora trovarsi nella rete, per fare ciò si assume che ho un pacchetto non possa "sopravvivere" nella rete per un periodo superiore a un lasso di tempo fissato

3.5 - Trasporto orientato alla connessione: TCP

Abbiamo già introdotto e più volte le ripreso TCP, descrivendolo come un protocollo di Internet a livello di trasporto:

- affidabile e
- orientato alla connessione

> **Ripasso**
> Si intende che il client e il server si scambiano informazioni di controllo a livello di trasporto prima che i messaggi a livello di applicazione comincino a fluire. Questa procedura viene detta handshaking e serve per mettere in allerta client e server preparandoli alla partenza dei pacchetti

Tuttavia, in questo paragrafo scopriremo anche che TCP si basa su molti dei concetti precedentemente trattati come:

- la rilevazione degli errori
- le ritrasmissioni
- gli ACK cumulativi
- i contatori e
- i campi nell'intestazione per:
  - i numeri di sequenza e
  - i numeri di ACK

Prima di andare a vedere la struttura dei segmenti TCP, ecco alcuni concetti chiave:

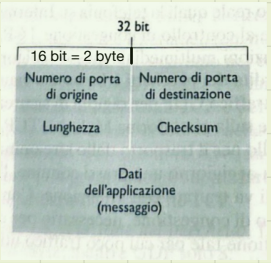
- una connessione TCP NON è un circuito end-to-end TDM o FDM, lo stato di connessione risiede completamente nei due sistemi periferici e non negli elementi di rete intermedi (router e switch a livello di collegamento) che non salvano lo stato di connessione e vedono solo datagrammi e non connessioni
- una connessione TCP offre un **servizio full-duplex**, ovvero su una connessione TCP tra un processo A su un host e un processo B su un altro host i dati a livello di applicazione possono fluire dal processo A al processo B nello stesso momento in cui fluiscono in direzione opposta
- una connessione TCP è anche detta **punto a punta** (point-to-point), ossia ha luogo tra un singolo mittente è un singolo destinatario (Multicast, trasferimento dati da un singolo mittente a più destinatari non possibile con TCP)
- la massima quantità di dati prelevabili e posizionabili in un segmento viene limitata dalla **dimensione massima di segmento** (MSS, Maximum Segment Size), un valore che viene generalmente impostato determinando prima la lunghezza del frame più grande, la cosiddetta **unità trasmissiva massima** (MTU, Maximum Transmission Unit) che può essere inviato a livello di collegamento dall'host mittente locale e poi scegliendo un MSS tale che il segmento TCP sti all'interno di un singolo frame a livello di collegamento

3.5.2 - Struttura dei segmenti TCP

I segmenti TCP presentano delle similarità con i segmenti UDP in termini di campi, difatti anch'essi hanno:

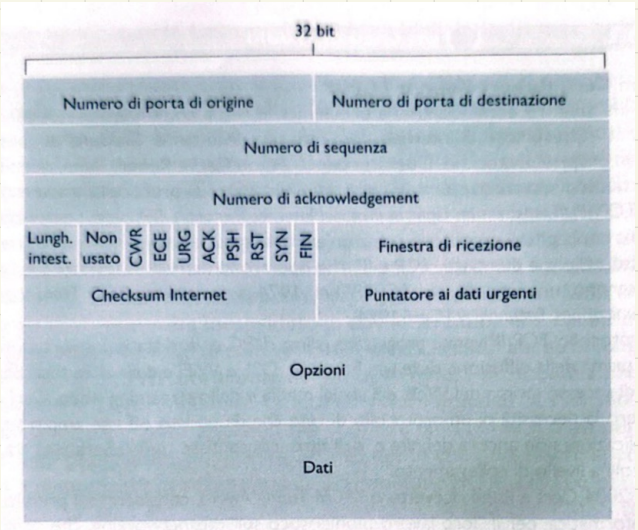
- Numero di porta di origine
- Numero di porta di destinazione e (per le operazioni di multiplexing e demultiplexing)
- Checksum (per verificare se sono avvenuti errori nel collegamento)

Ripasso
Intestazione UDP

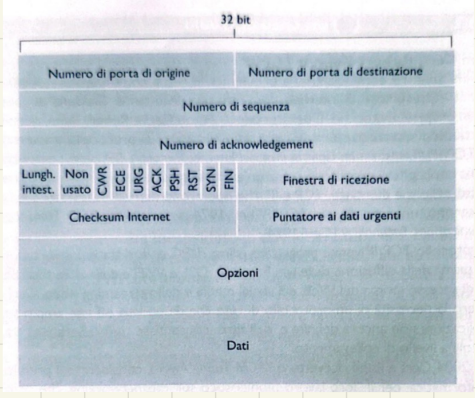


Oltre a questi ne sono presenti altri come:

- il campo **Numero di sequenza** (sequenze number field) e]
  il campo **Numero di acknowledgment** (acknowledgment number)] 32 bit
  vengono utilizzati dal mittente e dal destinatario TCP per implementare il trasferimento dati affidabile
- il campo **Finestra di ricerca** (receive window field)
  viene utilizzato per il controllo del flusso
- il campo **Lunghezza dell'intestazione** (header length field)
  specifica la lunghezza dell'intestazione TCP in multipli di 32 bit (generalmente vuoto => lunghezza consueta = 20 byte)
- il campo **Opzioni** (options)
  a differenza degli altri campi è facoltativo e di lunghezza variabile e viene utilizzato quando il mittente e il destinatario negoziano la dimensione massima del segmento (MSS) o come fattore di scala per la finestra nelle reti ad alta velocità + operazione di time-stamping



- il campo **Flag**] 6 bit
  - il bit **ACK** viene usato per indicare che il valore trasportato nel campo di acknowledgment è valido, ossia il segmento contiene un acknowledgment per un segmento che è stato ricevuto con successo
  - i bit **RST**, **SYN** e **FIN** vengono utilizzati per impostare chiudere la connessione
  - i bit **CWR** e **ECE** sono usati nel controllo di congestione esplicito
  - il bit **PSH** se ha valore 1, il destinatario dovrebbe inviare immediatamente i dati al livello superiore
  - il bit **URG** viene usato per indicare nel segmento la presenza di dati che vendita mittente livello superiore a marcato come "urgente"
- il campo **Puntatore dati urgenti** (urgent data pointer field)
  denota l'ultima posizione dell'ultimo byte di dati urgenti



(PSH, URG e il puntatore non vengono usati)

Dei campi appena trattati due dei più importanti sono il campo **Numero di sequenza** e il campo **Numero di acknowledgment** che come già detto servono per implementare il trasferimento dati affidabile

Partendo da i valori che TCP ci inserisce dentro, TCP vede i dati come un flusso di byte non strutturati, ma ordinati e l'uso dei numeri di sequenza in TCP riflette questa visione, dato che i numeri sequenza si applicano afflusso di byte trasmessi e non alla serie di segmenti trasmessi

Il N**umero di sequenza** è pertanto il numero nel flusso di byte del primo byte del segmento

In merito al **Numero di acknowledgment**, dato che TCP effettua l'acknowledgment solo dei byte fino al primo byte mancante nel flusso, si dice che tale protocollo offre **acknowledgment cumulativi** (cumulative acknoledgment)

3.5.3 - Timeout e stime nel tempo di andata e ritorno (RTT)

Come già anticipato, TCP gode di un contatore che gli permette di fare uso del meccanismo di timeout (tempo scaduto) dove, allo scadere del tempo, ritrasmette tutti i segmenti non confermati

Tuttavia, se in rdt l'implementazione era alquanto semplice, per TCP è un po' più problematica poiché, siccome il timeout dovrebbe essere più grande del tempo di andata e ritorno sulla connessione (**RTT, Round Trip Time**, tempo trascorso tra invio e ACK del segmento) per evitare ritrasmissioni inutili, viene da chiedersi di quanto il timeout dovrà essere grande

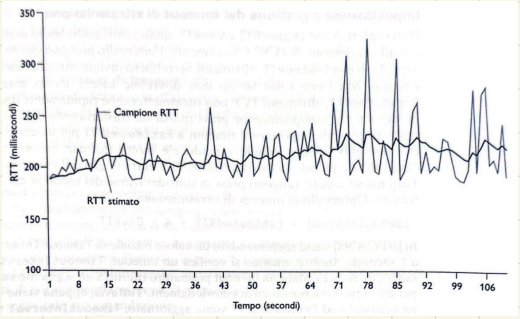
L'RTT misura di un segmento, denotato come "SampleRTT", e la quantità di tempo che intercorre tra l'istante di invio del segmento (ossia quando viene passato IP) e quello di ricezione dell'ACK del segmento

Anziché misurare un SampleRTT per ogni segmento, la maggior parte delle implementazioni TCP effettua una sola misurazione di SampleRTT alla volta, ovvero, in ogni istante di tempo, il SampleRTT viene valutato per uno solo dei segmenti trasmessi e per cui non si è ancora ricevuto ACK, il che comporta a prossimamente la misurazione di un nuovo valore di SampleRTT a ogni RTT

Inoltre, TCP non calcola mai il SampleRTT per i segmenti ritrasmessi, cioè lo calcola soltanto per i segmenti trasmessi una volta sola, tuttavia, ovviamente i campioni vanno da segmento a segmento in base alla congestione nei router e al diverso carico sui sistemi periferici e a causa di tale fluttuazione ogni valore di SampleRTT può essere atipico

Per effettuare una stima risulta naturale calcolare una media dei valori di SampleRTT, che TCP chiama “EstimatedRTT”, pertanto, quando si ottiene un nuovo SampleRTT, TCP aggiorna EstimatedRTT secondo la formula

$$EstimatedRTT = (1 - \alpha) \times EstimatedRTT + \alpha \times SampleRTT$$



Il nuovo valore di EstimatedRTT è una combinazione ponderata del suo precedente valore e del nuovo valore di SampleRTT, dove il valore raccomandato per alfa è 0,125

Oltre ad avere una stima di RTT, è anche importante possedere la misura della sua variabilità che definiremo come “DevRTT” che trattiamo come una stima di quanto SampleRTT si discosti da EstimatedRTT

$$DevRTT = (1 - \beta) \times DevRTT + \beta \times |\ SampleRTT - EstimatedRTT\ |$$

L’intervallo di Timeout non può essere inferiore a quello di EstimatedRTT, altrimenti verrebbero inviate ritrasmissioni non necessarie, ma l’intervallo stesso non dovrebbe essere molto maggiore di EstimatedRTT, altrimenti TCP non trasmetterebbe rapidamente il segmento perduto, il che comporterebbe gravi ritardi sul trasferimento dei dati

Pertanto auspicabile impostare il Timeout a EstimatedRTT + un certo margine che dovrebbe essere grande quando c’è molta frustrazione nei valori di SampleRTT e piccola in caso contrario, quindi entra in gioco il valore di DevRTT

Tutti questi aspetti vengono presi in considerazione dal modo in cui TCP determina l’intervallo di Timeout di trasmissione

$$TimeoutInterval = EstimatedRTT + 4 \times DevRTT$$

3.5.4 - Trasferimento dati affidabile

Di fronte al problema che i segmenti al livello di trasporto possono essere inglobati nei datagrammi IP che non garantisce la consegna nè l’integrità dei dati, TCP crea un **servizio di trasporto dati affidabile** (reliable data transfer service) al di sopra del servizio inaffidabile e best-effort di IP, in maniera tale che, oltre a risolvere i problemi di IP, non siano presenti duplicazione e venga rispettata la sequenza originaria, ovvero il flusso di dati in arrivo è esattamente quello spedito

Per garantire questo servizio TCP utilizza i timer di ritrasmissione che però, invece di associarne uno per ogni segmento trasmesso come visto precedentemente, ne usa solo uno anche in presenza di più segmenti trasmessi

Per comprendere tale funzionamento faremo uso di una descrizione semplificata di un mittente TCP, una in cui viene utilizzato solo il timeout e in cui i dati sono inviati solo un'una direzione, dal mittente al destinatario

Esistono tre venti principali alla trasmissione e ritrasmissione dei dati:

1. **Dati provenienti dall'applicazione**
   TCP incapsula i dati che li giungono all'applicazione in un segmento e lo passa a IP, segmenti che includono un numero di sequenze che rappresenti il numero del primo byte dai il segmento nel flusso di byte

   Peraltro, se il timer non è già in funzione per qualche altro segmento, TCP lo avvia quando il segmento viene passato IP ed è utile pensare che il timer sia associato al più vecchio segmento che non ha ricevuto acknowledgment e di cui l'intervallo di scadenza è detto "TimeoutInterval"

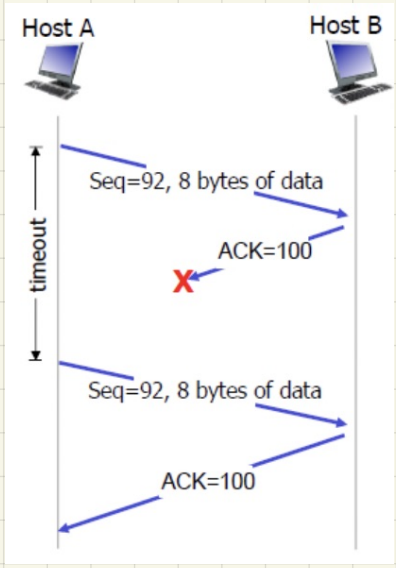
2. **Timeout**
   Molto semplicemente, quando il timeout si verifica, TCP risponde ritrasmettendo il segmento che lo ha causato e quindi riavviando il timer

3. **Ricezione di un ACK**
   Consiste nella ricezione di un ACK con un valore valido nel campo ACK

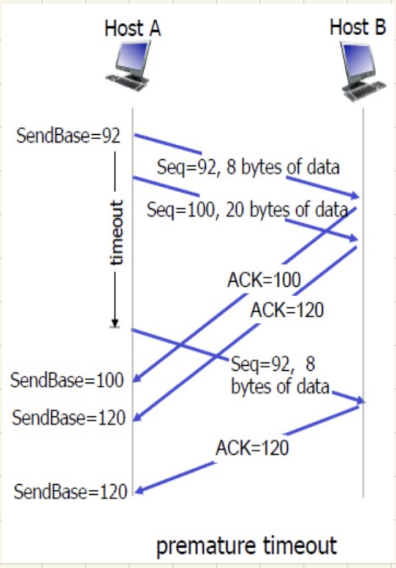
   Quando si verifica, TCP confronta il valore *y* di ACK con la propria variabile "SendBase", una variabile di Stato che è il numero di seguenze del più vecchio byte che non ha ancora ricevuto un ACK, pertanto il numero di seguenze dell'ultimo byte che si sa essere stato ricevuto correttamente nell'ordine giusto è pari a SendBase - 1

   E siccome TCP utilizza ACK cumulativi, *y* conferma la ricezione di tutti i byte precedenti al byte numero *y*, ma se *y* è maggiore di SendBase, allora l'ACK si riferisce a uno o più segmenti che precedentemente non avevano ancora ricevuto conferma, quindi il mittente aggiorna la propria variabile SendBase e poi, se non ci sono i segmenti che ancora necessitano di ACK, riavvia il timer

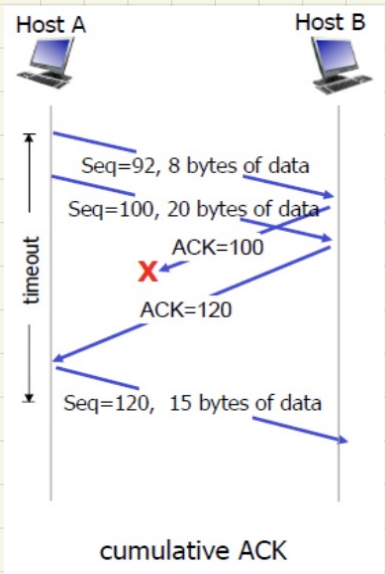
Di seguito alcuni scenari pratici



Un segmento viene inviato e consegnato correttamente, ma non viene confermato perché l'ACK si è perso, allora il mittente ritrasmette, ma siccome destinatario lo ha già ricevuto lo scarta



Due segmenti vengono inviati e consegnati correttamente, ma vengono confermati dopo che si è verificato il timeout che porterà il mittente a rinviare solo il primo segmento e non il secondo almeno finché del primo non avrà ricevuto l'ACK prima del nuovo timeout



Due segmenti vengono inviati e consegnati correttamente, ma solo il secondo viene confermato e siccome TCP fa uso di ACK duplicativi il mittente capisce che è pure arrivato il primo pacchetto, anche se non confermato dall'ACK associato, quindi non li ritrasmette

Altre varianti in merito al timeout sono:

- **Raddoppio dell'intervallo di timeout**
  Con questa modifica allo scadere del time out, TCP ritrasmette segmento con il più basso numero di sequenza che non abbia ricevuto ACK, tuttavia impoSTA il successivo intervallo di Timeout al doppio del valore precedente anziché derivarlo dagli ultimi EstimatedRTT e DevRTT

  Facendo così però gli intervalli cresceranno esponenzialmente ad ogni ritrasmissione, offrendo così una forma limitata di controllo di congestione

  Congestione che peraltro può essere la causa della scadenza del timer e può essere causata da pacchetti in arrivo da una o più code di router e a ragione del fatto che se la rete è congestionata, molti dei pacchetti in transito possono andare incontro a lunghi ritardi di accodamento e/o eliminazione

  A differenza dei router, TCP si comporta in maniera più rispettoso della rete, dato che ciascun mittente ritrasmette dopo intervalli sempre più lunghi

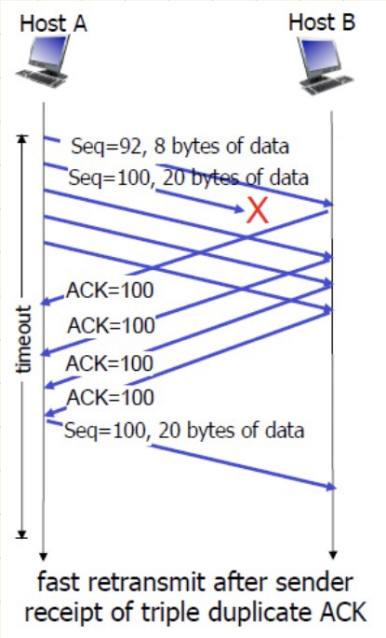
- **Ritrasmissione rapida**

  Uno dei problemi legati alla ritrasmissione e che il peso di Timeout può rilevarsi relativamente lungo e se così, nel caso in cui un segmento si sia smarrito, il mittente dovrà ritardare la ritrasmissione del pacchetto smarrito

  Tuttavia, siccome TCP può utilizzare gli **ACK duplicati**, grazie ad essi il mittente può rendersi conto se un pacchetto sia stato smarrito prima dell'effettivo scadere del timeout

  Quando il destinatario riceve un segmento con numero di sequenza superiore al successivo numero di sequenza atteso in ordine, il destinatario rileva un buco nel flusso di dati, un segmento mancante e siccome non può mandare un ACK negativo (NACK) perché il TCP non lo supporta, si limita a mandare nuovamente un ACK relativo all'ultimo byte che ha ricevuto in ordine, duplicando così un ACK

  E siccome il mittente può mandare più segmenti alla volta, se uno di questi si smarrisce, allora ci saranno più ACK duplicati e nel caso in cui mittente ne riceva tre per lo stesso dato, considererà questo evento come indice che il successivo segmento sia andato perduto ed effettuerà una **ritrasmissione rapida** (fast retransmit), ovvero rispedire il segmento mancante prima che scada il timer



NOTA - TCP è GBN o SR?

Considerando che TCP fa uso di ACK cumulativi e che i segmenti ricevuti correttamente, in modo disordinato, non vengono notificati singolarmente al destinatario, il mittente deve solo memorizzare il numero di sequenze più basso tra i byte trasmessi che non hanno ancora ricevuto ACK il numero di seguenze del successivo byte da inviare, rendendolo simile a un protocollo GBN

Tuttavia il destinatario memorizza nel suo buffer i segmenti ricevuti correttamente, ma non in ordine (come anche visto precedentemente), rendendolo simile a un protocollo SR

Quindi alla fine TCP può essere classificato come un ibrido tra i protocolli di tipo GBN e SR

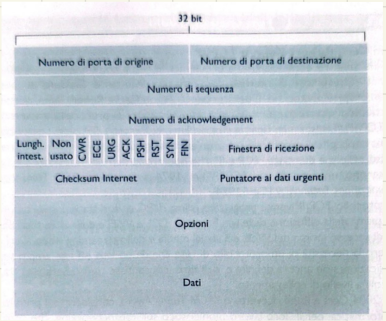
Se TCP fosse dotato di ACK selettivo (selective ACK), dando la possibilità destinatario di mandare ACK per i segmenti non in ordine anziché cumulativamente e fosse dotato anche di trasmissione selettiva, sarebbe più simile a un generico protocollo SR

3.5.5 - Controllo di flusso

Durante la stesura degli argomenti precedenti abbiamo omesso un particolare che potrebbe verificarsi, ovvero quelli in cui l'applicazione ricevente potrebbe essere impegnata in qualche altro compito e potrebbe non leggere i dati in parallelo al loro arrivo, comportando un overflow del buffer dì ricezione

In risposta a ciò TCP offro un **servizio di controllo di flusso** (flow-control service) che consiste in un servizio di confronto sulla velocità, dato che paragona la frequenza di invio del mittente con quella di lettura dell'applicazione ricevente

DA NON CONFONDERE CON IL **CONTROLLO DI CONGESTIONE** DA PARTE DEL MITTENTE



Il servizio di controllo del flusso è offerto facendo a mantenere al mittente una variabile chiamata **finestra di ricezione** (receive window), la quale fornisce al mittente l'indicazione dello spazio libero disponibile nel buffer del destinatario

In un contesto di trasferimento di file, supponiamo che un host A stiamo inviando un file di grandi dimensioni a un host B suona connessione TCP

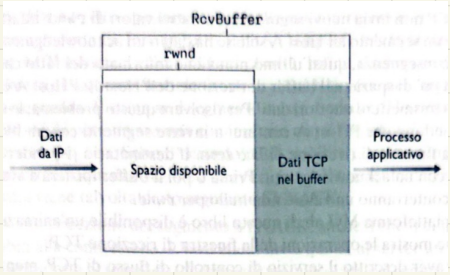
L'host B allocca un buffer di ricezione per la connessione, la cui dimensione è denotata RcvBuffer, mentre il suo processo applicativo legge dal buffer

Ora definiamo le seguenti variabili:

- LastByteRead: numero dell'ultimo byte nel flusso di dati che il processo applicativo in B ha letto dal buffer
- LastByteRcvd: numero dell'ultimo byte, nel flusso di dati, che proviene dalla rete e che è stato copiato nel buffer di rielezione di B

Siccome TCP non può mandare in overflow il buffer allocato, dovremmo per forza avere

RcvBuffer => LastByteRcvd - LastByteRead



La finestra di ricezione, indicata con rwnd, viene impostata la quantità di spazio disponibile nel buffer, pertanto

rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]

L'host B comunicherà all'host A quanto spazio disponibile c'è nel buffer della connessione scrivendo il valore corrente di rwnd nel campo apposito dei segmenti che manda ad A e sempre l'host B inizializza rwnd con il valore RcvBuffer tenendo traccia di variabili specifiche per ogni connessione

A sua volta l'host A tiene traccia di due variabili

- LastByteSent, ultimo byte mandato
- LastByteRead, ultimo byte letto (ultimo byte per cui si è ricevuto ACK)

La cui differenza esprime la quantità di dati spediti da A per cui non si è ancora ricevuto un ACK che peraltro deve rimanere sempre minore di rwnd così da non mandare in overflow il buffer di recisione dell'host B, quindi l'host A deve rispettare la disuguaglianza

rwnd => LastByteRcvd - LastByteRead

Tuttavia in questo schema c'è un problema tecnico

Supponendo che rwnd = 0 e che l'host A non abbia più nulla da trasmettere, dopo che in B il processo applicativo avrà svuotato il buffer, TCP non invierà nuovi segmenti con nuovi valori di rwnd, perché TCP fa pervenire un segmento all'host A solo se ha dati o un ACK da mandare, di conseguenza non verrà informato del fatto che si è liberato dello spazio nel buffer di ricezione e quindi rimarrà bloccato e non potrà trasmettere ulteriori dati

Per risolvere questo problema, le specifiche TCP richiedono che l'host A continua inviare segmenti con un paio di dati quando la finestra ricezione di B è zero, così facendo destinatario risponderà a questi segmenti con un ACK e prima o poi il buffer inizierà a svuotarsi riscontri conterranno un valore non nullo per rwnd

NOTA - UDP non fornisce controllo del flusso

3.5.6 - Gestione della connessione TCP

Diamo ora un'occhiata a come viene stabilita e rilasciata una connessione TCP

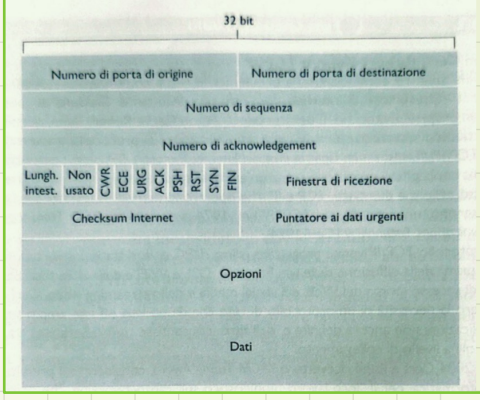
- **Stabilire la connessione**

  Supponiamo che un processo di esecuzione in un host (client) voglia inizializzare una connessione con un altro processo in esecuzione in un altro host (server)

  Il processo applicativo client dapprima informa il lato client di TCP che vuole stabilire una connessione verso un processo nel server, per poi il lato client di TCP procede a stabilire una connessione TCP con il TCP nel server del mondo descritto di seguito:

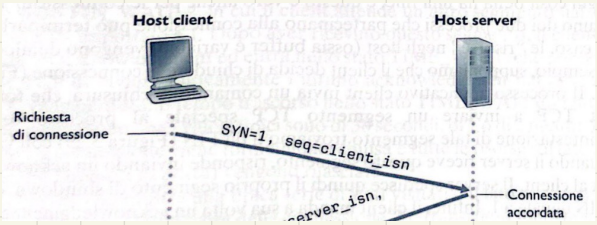
Ripasso

i bit **RST**, **SYN** e **FIN** vengono utilizzati per impostare chiudere la connessione



1. **TCP lato client invia uno speciale segmento al TCP lato server**

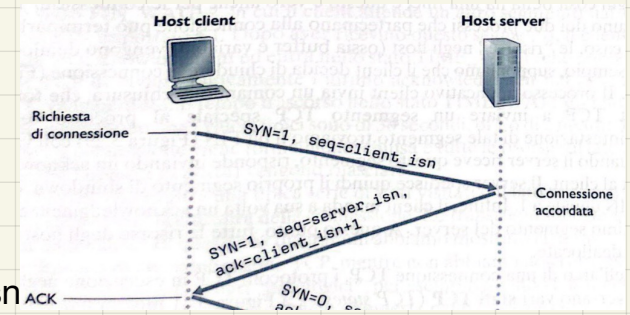
   Questo segmento speciale non contiene dati a livello applicativo, ma uno dei bit di intestazione del segmento, il bit SYN che è posto a 1, nonché motivo per cui tale segmento viene detto **segmento SYN**



   Inoltre il client sceglie a caso un numero di sequenza iniziale (client_isn) e lo pone nel campo numero di sequenza del segmento SYN iniziale che viene incapsulato in un datagramma IP e inviato al server

2. **Quando il datagramma IP arriva all'host server**

   Il server estrae il segmento dal telegramma, alloca il buffer e le variabili TCP alla connessione e invia un segmento di connessione approvata al TCP



   Anche questo segmento non contiene dati a livello applicativo, ma nella sua intestazione ci sono 3 informazioni importanti:

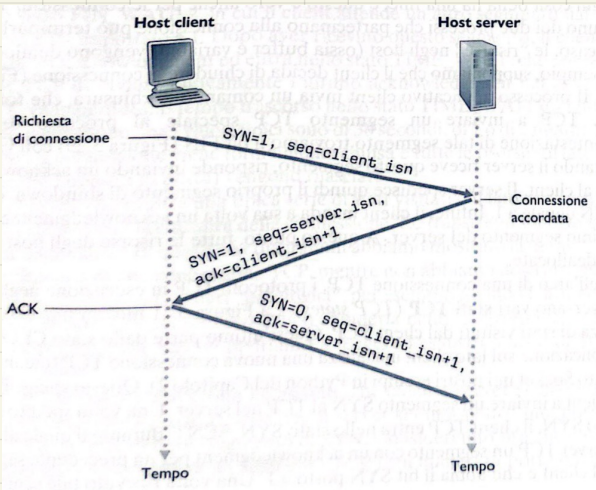
   - SYN = 1
   - il campo ACK = client_isn + 1
   - il campo numero di sequenza = server_isn (numero di sequenza del server)

   Siccome il segmento utilizza sia SYN che ACK viene chiamato anche **segmento SYNACK**

3. **Ricezione segmento SYNACK**

   Anche il cliente alloca buffer e variabili alla connessione e quindi invia al server un altro segmento in risposta al segmento di connessione approvata dal server, operazione:

   - che viene svolta ponendo il campo ACK = server_isn + 1 e
   - in cui il campo SYN = 0, dato che la connessione è stata stabilita



   In questo terzo passo il campo dati del segmento può contenere informazioni che vanno dal cliente verso il server in cui il campo SYN = 0

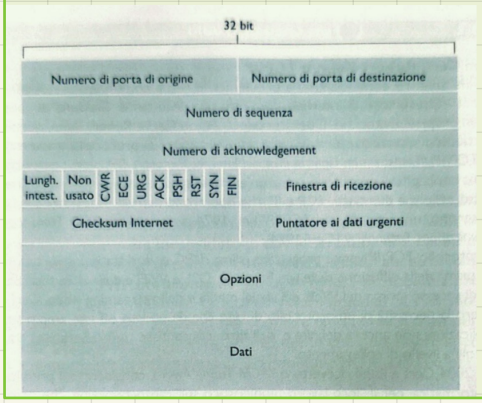
   Il modo in cui è strutturata questa procedura le è valso il nome di **handshake a tre vie**

- **Rilasciare la connessione**

  Per chiudere la connessione ciascuno dei processi che partecipano alla connessione può terminarla, con il conseguente deallocamento delle risorse negli host (buffer e variabili)

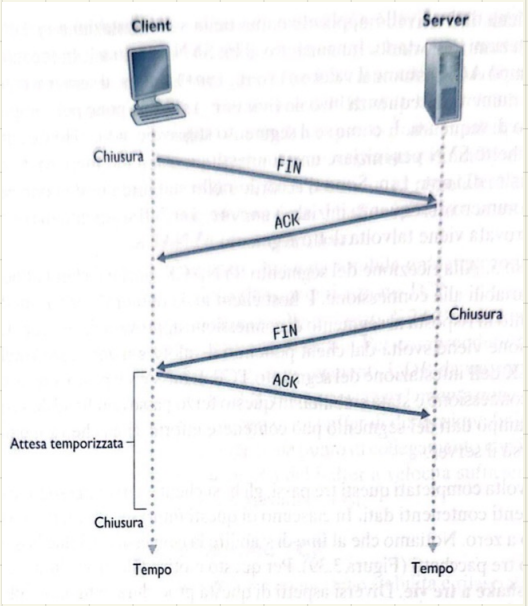
Ripasso

i bit **RST**, **SYN** e **FIN** vengono utilizzati per impostare chiudere la connessione



Supponiamo che un processo di esecuzione in un host (client) voglia chiudere la connessione

Il processo applicativo client invia un comando di chiusura che forza il client TCP a inviare un segmento TCP speciale al processo server, un segmento dove il bit FIN = 1

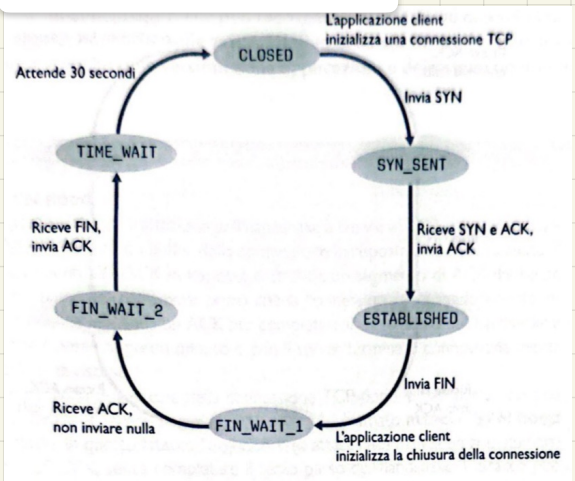


In risposta a ciò, il server risponde inviando un ACK al client spedendo il proprio segmento di Shutdown con il bit FIN = 1

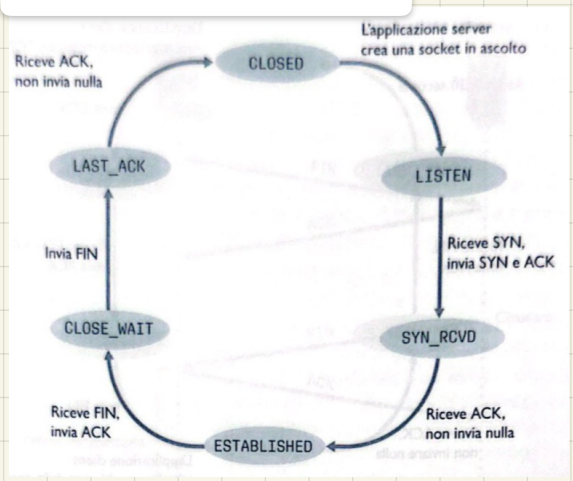
Infine, il client manda a sua volta un ACK a quest'ultimo segmento del server e a questo punto tutte le risorse degli host risultano deallocate

Nell'arco di una connessione TCP, i protocolli TCP in esecuzione negli host attraversano vari **stati TCP** (TCP states), raffigurati nelle immagini a seguire

Stati TCP visti dal client



Stati TCP visti dal server



NOTA - Attacco SYN flood

Un tipico attacco di tipo DOS (Denial of Service) che consiste nel mandare un gran numero di segmenti TCP SYN senza completare il terzo passo dell'handshake

3.6 - Principi del controllo di congestione

Oltre al controllo del flusso, un altro problema a cui il protocollo TCP va incontro è quello della congestione della rete, una situazione in cui più sorgenti provano a inviare dati a ritmi troppo elevati, con la possibilità che questi vengono smarriti o peggio arrivano in ritardo con conseguenti ritrasmissioni

Ma come già anticipato, TCP fornisce un servizio di controllo della congestione, che adegua l'attività dei mittenti in relazione al traffico

3.6.1 - Cause e costi della congestione

Molto velocemente vediamo alcuni concetti sulle cause e costi della congestione:

- il throughput non può mai superare la capacità della rete
- i ritardi aumentano all'approcciarsi alla capacità massima
- perdite/ritrasmissioni diminuiscono il throughput effettivo
- duplicati non richiesti diminuiscono ancora di più il throughput effettivo

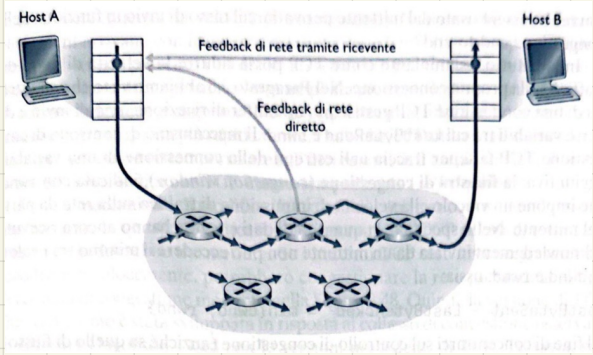
Ripasso

Throughput: quantità di dati al secondo che può essere trasferita tra due sistemi periferici

3.6.2 - Approcci al controllo di congestione

In seguito riprenderemo in maniera approfondita l'approccio specifico di TCP al controllo di congestione, per ora vediamo i due principali orientamenti utilizzati

- **Controllo di congestione end-to-end**
  Siccome il livello di rete non fornisce supporto esplicito al livello di trasporto per il controllo di congestione, questa viene dedotta in base all'osservazione del comportamento della rete, se ci sono ritardi o perdite di bacchetti, in tal caso TCP diminuisce l'ampiezza della propria finestra
  (Nuove proposte di TCP fanno uso di valori crescenti di RTT come indicatori di traffico sempre più intenso sulla rete)



- **Controllo di congestione assistito dalla rete**
  I componenti livello di rete (i router) forniscono un feedback esplicito al mittente sullo stato di congestione della rete

3.7 - Controllo di congestione TCP

Come già anticipato precedentemente, TCP utilizza il controllo di congestione end-to-end, invece che quello assistito dalla rete, proprio perché il livello di rete non fornisce sistemi periferici alcun feedback esplicito per quanto riguarda la congestione della rete

In merito al controllo di congestione TCP, ne esistono varie versioni a seconda della versione di TCP, ma noi quella che tratterremo sarà quella del TCP classico

3.7.1 - Controllo di congestione di TCP classico

L'approccio scelto da TCP consiste nell'imporre a ciascun mittente un limite al tasto di invio sulla propria connessione in funzione della concessione di rete percepita, dove:

- se il mittente TCP si accorge di condizioni di scarso traffico sul percorso che porta alla destinazione, incrementa il proprio tasso di trasmissione
- se il mittente percepisce traffico lungo il percorso, lo riduce

Quanto trattato in maniera alquanto superficiale prende il nome di **algoritmo di controllo di congestione di TCP** (TCP congestione-control algorithm) ed è strutturato in tre fasi:

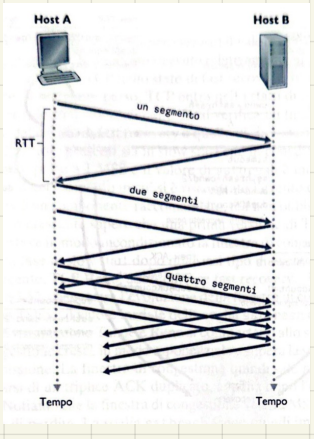
1. slow start
2. congestion avoidance
3. fast recovery

Slow start e congestion avoidance sono componenti obbligatorie di TCP e differiscono nel modo in cui aumentano la grandezza di cwnd in risposta agli ACK ricevuti, anche se slow start incrementa cwnd molto più rapidamente di congestion avoidance

La fast recovery è suggerita, ma non obbligatoria, per i mittenti TCP

1. **Slow start**

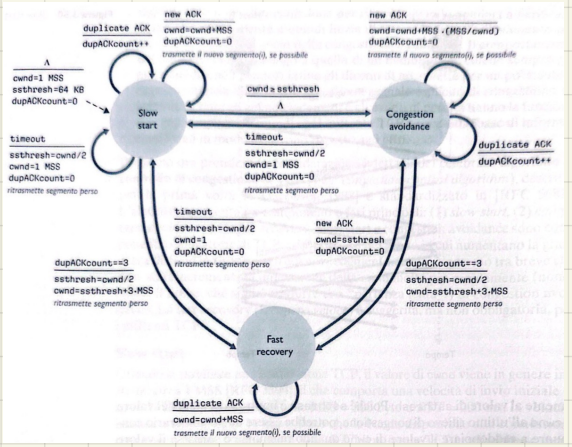
   Quando si stabilisce una connessione TCP, il valore di cwnd vieni in genere inizializzato a 1 MSS, con una velocità di invio iniziale pari a MSS/RTT, ma dato che la banda disponibile può essere molto più grande di questo valore, il valore di cwnd può essere incrementato e così viene fatto, viene duplicato ogni volta che un segmento trasmesso riceve l'ACK, cioè ad ogni RTT così facendo, la velocità di trasmissione parte lentamente, ma cresce esponenzialmente



Tuttavia questa crescita termina nel momento in cui:

- avviene un episodio di perdita di un pacchetto, indicato da un evento di Timeout

  In tal caso il mittente mette pone:
  - cwnd = 1 nuovamente riniziando la fase di slow start
  - una seconda variabile di stato detta soglia di slow start uguale a cwnd/2



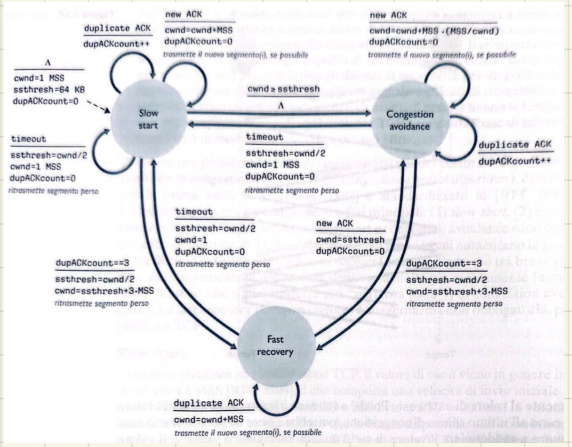
- cwnd = soglia di slow start
  e in aggiunta TCP entra in modalità di congestion avoidance

2. Congestion avoidance

Quando TCP entro in questa fase, il valore di cwnd è circa la metà di quello che aveva l'ultima volta in cui è stata rilevata la congestione

Invece di raddoppiare il valore di cwnd ogni RTT, TCP incrementa cwnd di 1 MSS ogni RTT

Tale algoritmo termina l'incremento lineare quando si verifica un Timeout, a quel punto cwnd e posto uguale a 1 MSS è il valore della soglia viene impostato alla metà del valore di cwnd (Treshold) al momento del Timeout



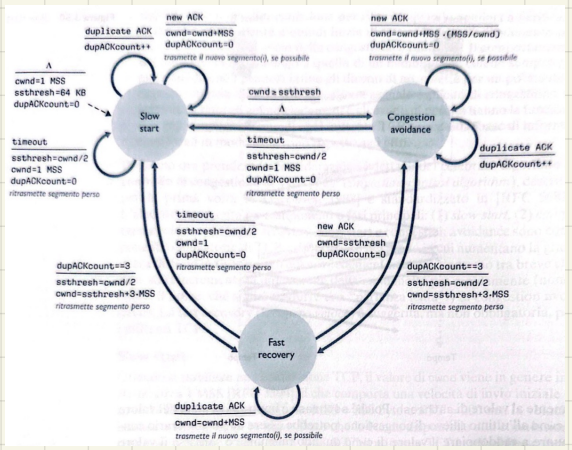
3. Fast recovery

Durante tale fase il valore di cwnd è incrementato di 1 MSS per ogni ACK duplicato ricevuto relativamente al segmento perso che ha causato l'entrata di TCP nello stato di Fast recovery

Infine, quando arriva un ACK per il segmento perso, TCP entra nello stato di congestion avoidance dopo aver ridotto il valore di cwnd

Se si verifica un Timeout vi è invece una transizione dallo stato di Fast recovery a quello di slow start dopo aver effettuato le stesse azioni presenti sia in slow start che in congestion avoidance

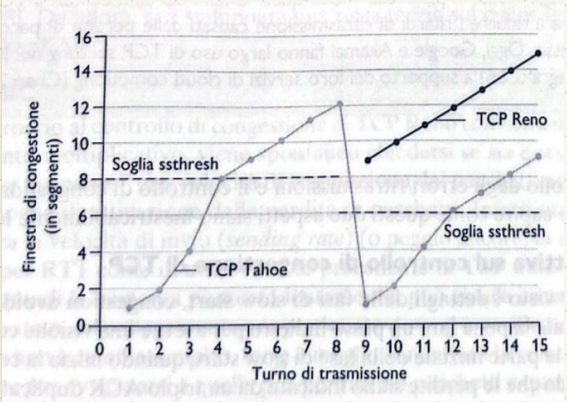
- cwnd = 1 nuovamente riniziando la fase di slow start
- una seconda variabile di stato detta soglia di slow start uguale a cwnd/2



Nell'immagine a fianco vediamo l'andamento di due versioni differenti di TCP

- **TCP Tahoe**
  Questa versione passata portava in modo incondizionato la finestra di congestione a 1 MSS ed entrava nella fase di Slow Stat dopo qualsiasi tipo evento di perdita

- **TCP Reno**
  Questa versione recente adotta fast recovery

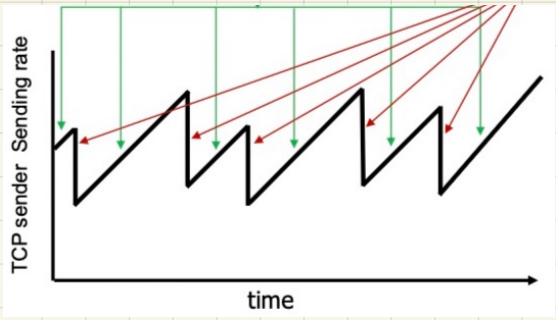


Il sistema appena descritto prende il nome di AIMD (Additive Increase Multiplicative Decrease) e di questo si distinguono:

- Incremento additivo
  aumenta il rate di invio di 1 MSS ogni RTT, fino a quando non viene rilevata la perdita
- Decremento moltiplicativo, dove:
  - dimezza il rate di invio ad ogni perdita segnalato da Triplo ACK duplicato (TCP reno)
  - riporta il rate di invio a 1 MSS ad ogni perdita segnalata da TCP (TCP Tahoe)

NOTA - AIMD comportamento a fisarmonica

L'algoritmo è distribuito e asincrono per ottimizzare la stabilità



In pratica TCP si adatta automaticamente, assumendo un comportamento che consiste nell'inviare 1 byte e aspettare per ACK e se tutto va bene aumenta e invia più byte

Tutto questo è regolato dalle seguenti formule

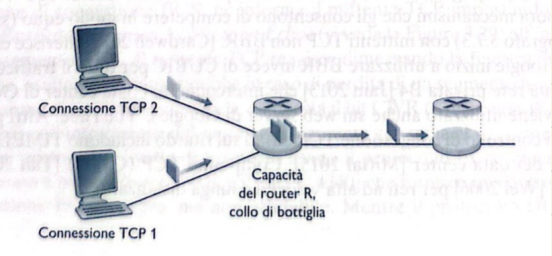
$$\text{TCP rate} = \frac{\text{cwnd}}{\text{RTT}} \text{ byte/sec}$$

Anche la window di invio si aggiustata dinamicamente in risposta alla congestione della rete

cwnd >= LastBySent - LastByAcked

3.7.3 - Fairness

Consideriamo K connessione TCP, ciascuna con un differente percorso end-to-end, ma che passano tutti attraverso un collegamento con capacità trasmissiva R bps che costituisce il collo di bottiglia del sistema, cioè tutti gli altri collegamenti non sono congestionati dispongo di elevate capacità trasmissiva in confronto a quella del collegamento che fa da collo di bottiglia



Si dice che un meccanismo di controllo di concessione sia equo (fair) se la velocità trasmissiva media in ciascuna connessione è approssimativamente R/K, ovvero ciascuna connessione ottiene la stessa porzione di banda del collegamento

La fairness è implementata in tutti le applicazioni TCP e si regolano tutte automaticamente, tuttavia non sempre è comoda, specie delle app multimediali che risulta usano UDP poiché non vogliono che il rate venga limitato, tollerare la perdita di pacchetti, ma devono avere audio/video costanti

In termini di controllo della congestione per UDP, se questi creano troppa congestione, i pacchetti vengono scartati

Ripasso CAP 2 pag 5

- **Temporizzazione**

  Di fronte alla necessità che ogni bit che il mittente invia sulla socket venga ricevuto dalla socket di destinazione in un determinato intervallo di tempo, un protocollo a livello di trasporto potrebbe fornire garanzie di temporizzazione (timing)

Il timing interessa:

- telefonia Internet
- ambienti virtuali
- teleconferenze
- giochi multiutente